

I. Portada

APRENDIZAJE DE MÁQUINAS (ACIF104) INFORME FASE 2

Nombre integrante/s:

Isabel Vera

Gabriel Vera

Diego Mallea

Curso: ACIF104 - Taller de Machine Learning

Fecha: 28/08/2026

II. Problemática

2.1. Problemática seleccionada

Las aplicaciones digitales y los servicios por suscripción enfrentan un desafío relevante asociado a la retención de usuarios. En este tipo de negocios, el abandono de clientes, también conocido como churn, ocurre cuando una persona deja de utilizar un servicio, cancela su suscripción o reduce su nivel de interacción hasta dejar de generar valor para la organización.

El problema principal es que muchas organizaciones detectan el abandono cuando ya ocurrió, es decir, cuando el cliente ya canceló, dejó de ingresar a la plataforma o dejó de pagar. Esta reacción tardía limita la posibilidad de ejecutar acciones preventivas, como campañas de retención, mejoras en la experiencia de usuario, contacto desde soporte, beneficios personalizados o recomendaciones de uso.

En el contexto de una aplicación digital, el comportamiento de los usuarios genera múltiples datos que pueden ser utilizados para anticipar el riesgo de abandono. Algunos ejemplos son la frecuencia de uso, la cantidad de sesiones, los días desde el último ingreso, el tipo de plan contratado, la antigüedad del usuario, el método de pago, las interacciones con soporte y el nivel de engagement. Estos datos permiten construir un modelo de aprendizaje automático capaz de identificar patrones asociados al abandono.

Por lo tanto, la problemática seleccionada corresponde a la necesidad de predecir el riesgo de abandono de clientes en una aplicación digital, utilizando técnicas de aprendizaje automático supervisado. La solución propuesta busca apoyar la toma de decisiones organizacionales mediante una aplicación que permita estimar si un usuario presenta alto o bajo riesgo de abandono.

Dataset seleccionado:
Para el desarrollo del proyecto se seleccionó el dataset Customer Churn Prediction Business Dataset, disponible en Kaggle (Miah, 2025). Este conjunto de datos es sintético, pero está diseñado para representar un escenario empresarial realista de predicción de abandono de clientes, por lo que resulta pertinente para el alcance académico del proyecto

2.2. Contexto organizacional, técnico, económico y social

Desde el punto de vista organizacional, la predicción de abandono permite que las empresas pasen de una gestión reactiva a una gestión preventiva. En lugar de actuar cuando el cliente ya canceló el servicio, la organización puede identificar señales tempranas de riesgo y priorizar acciones de retención.

Desde el punto de vista técnico, el problema es abordable mediante aprendizaje supervisado, ya que se cuenta con una variable objetivo que indica si el cliente abandonó o no el servicio. Este tipo de aprendizaje utiliza datos etiquetados para entrenar modelos capaces de predecir nuevos casos a partir de patrones históricos.

Desde el punto de vista económico, el abandono de clientes afecta directamente los ingresos recurrentes, la estabilidad financiera y la eficiencia de las estrategias comerciales. Además, distintos trabajos previos indican que retener clientes existentes suele ser más eficiente que adquirir nuevos usuarios, por lo que anticipar el churn puede tener impacto directo en la rentabilidad de la organización.

Desde el punto de vista social y ético, una solución de este tipo debe ser cuidadosamente diseñada, ya que trabaja con datos de comportamiento de usuarios. Por ello, resulta necesario considerar requisitos de privacidad, anonimización, seguridad, explicabilidad y transparencia. El sistema no debe utilizar variables sensibles ni generar decisiones automatizadas que afecten injustamente a las personas.

2.3. Análisis de trabajos previos

La literatura reciente muestra que la predicción de abandono de clientes es un problema vigente y ampliamente estudiado en sectores como telecomunicaciones, banca, comercio electrónico, educación y servicios por suscripción. Imani et al. (2025) realizaron una revisión sistemática de estudios publicados entre 2020 y 2024, identificando que los métodos de Machine Learning y Deep Learning son cada vez más utilizados para predecir churn. Esta revisión evidencia que el problema sigue siendo relevante debido a su impacto en los ingresos, la fidelización y la toma de decisiones estratégicas.

Maan y Maan (2023) proponen un modelo de predicción de churn basado en aprendizaje automático explicable. Su trabajo destaca que no basta con predecir qué clientes tienen mayor riesgo de abandono, sino que también es necesario explicar qué variables influyen en la predicción. Este antecedente es especialmente relevante para el presente proyecto, ya que la solución propuesta debe entregar resultados comprensibles para usuarios no técnicos.

Shaikhsurab y Magadum (2024) estudiaron un enfoque de ensemble learning adaptativo para la predicción de churn en telecomunicaciones. Su propuesta integra modelos como XGBoost, LightGBM, LSTM, MLP y SVM, combinados mediante técnicas de stacking. Este trabajo muestra que los modelos avanzados pueden mejorar el desempeño predictivo, aunque también aumentan la complejidad computacional y reducen la interpretabilidad.

Alotaibi y Haq (2024) compararon distintos clasificadores para predecir abandono de clientes en telecomunicaciones, incluyendo Random Forest, LightGBM, XGBoost, regresión logística, árboles de decisión y redes neuronales artificiales. El estudio refuerza la importancia de comparar múltiples técnicas antes de seleccionar el modelo final, ya que el rendimiento puede variar según el dataset, el balance de clases y las variables disponibles. Vafeiadis et al. (2015) llegaron a conclusiones similares al comparar árboles de decisión, SVM, redes bayesianas y k-vecinos más cercanos en este mismo problema, reforzando la conveniencia de comparar múltiples algoritmos antes de elegir un modelo final.

Bhattacharjee et al. (2023) abordaron la predicción temprana de churn a partir de series temporales de interacción usuario-producto. Este trabajo es relevante porque demuestra que el comportamiento de uso de los usuarios puede contener señales tempranas de abandono. Aunque el presente proyecto utilizará un dataset tabular y no necesariamente una serie temporal compleja, el estudio sirve como antecedente para justificar el uso de variables relacionadas con frecuencia de uso, antigüedad e interacción.

Por otra parte, Nogare y Silveira (2024) destacan la importancia de MLOps para gestionar el ciclo de vida de los modelos de Machine Learning, desde la experimentación hasta el despliegue y monitoreo. Este antecedente es importante para incorporar requisitos de mantenibilidad y monitoreabilidad, ya que un modelo predictivo puede perder rendimiento con el tiempo si cambian los patrones de uso de los clientes.

A partir del análisis de estos trabajos, se concluye que la predicción de churn es una problemática relevante y vigente. Además, se identifican tres necesidades centrales para el proyecto: construir un modelo predictivo de clasificación binaria, comparar distintas técnicas de aprendizaje automático y diseñar una solución que sea explicable, usable y monitoreable.

2.4. Alcance del proyecto

El alcance inicial del proyecto considera el desarrollo de una aplicación prototipo capaz de predecir si un cliente presenta riesgo de abandono a partir de datos históricos. Para ello, se utilizará un dataset de Kaggle relacionado con churn de clientes, que incluya variables numéricas y categóricas.

El proyecto contempla:

- Carga y análisis exploratorio del dataset.
- Limpieza de datos.
- Codificación de variables categóricas.
- Separación de datos en entrenamiento y prueba.
- Entrenamiento de al menos tres modelos de clasificación.
- Comparación de modelos mediante métricas objetivas.
- Desarrollo de una interfaz inicial para ingresar datos de un cliente y obtener una predicción.
- Incorporación de una explicación básica del resultado del modelo.

Queda fuera del alcance de esta fase la implementación productiva en una empresa real, la integración con bases de datos empresariales, el uso de datos personales reales y la automatización de campañas de retención.

III. Metodología

El proyecto se desarrolló mediante un proceso iterativo inspirado en CRISP-DM, adaptado al alcance académico. Las fases ejecutadas produjeron artefactos verificables en GitHub y utilizaron el mismo criterio de negocio: identificar la mayor cantidad posible de clientes con riesgo de abandono, sin perder de vista la precisión, el F1 y el costo de las falsas alarmas.

3.1 Objetivos operativos

- O1. Preparar y analizar el dataset, asegurando completitud, consistencia y ausencia de filtración de información.
- O2. Comparar tres modelos de machine learning y tres arquitecturas de deep learning bajo métricas comunes.
- O3. Seleccionar y refinar una solución orientada a detectar churn, considerando recall, F1, ROC-AUC y PR-AUC.
- O4. Integrar el modelo en un prototipo explicable, usable, reproducible y monitoreable.

3.2 Fases metodológicas

Análisis exploratorio y preparación. Se revisaron las 32 variables, la completitud, los duplicados, la consistencia categórica, la correctitud numérica y los valores atípicos. Luego se separaron entrenamiento, validación y prueba de forma estratificada, y el preprocesador se ajustó solo con entrenamiento.

Selección de técnicas. Se escogieron regresión logística, Random Forest y XGBoost como modelos supervisados para datos tabulares. La comparación se amplió con tres redes: MLP superficial, MLP profunda y MLP residual. Las redes se entrenan mediante retropropagación del error (Rumelhart et al., 1986), y su comparación frente a modelos de árboles responde a evidencia reciente de que las redes neuronales no siempre superan a los métodos basados en árboles en datos tabulares (Borisov et al., 2024).

Diseño de arquitectura. La entrada quedó compuesta por 58 características transformadas y la salida por una probabilidad binaria de churn. Se definieron configuraciones moderadas y reproducibles para CPU, con regularización y detención temprana en las redes.

Implementación, validación y evaluación. Se entrenaron 12 escenarios de balanceo para los modelos de ML y tres arquitecturas de DL. La selección se realizó sobre validación y XGBoost + ROS se evaluó posteriormente sobre 1.500 casos de prueba.

Explicabilidad e integración. Se incorporaron SHAP, un backend de inferencia, una interfaz en Streamlit y un registro de predicciones, completando la integración del prototipo.

3.3 Plan de trabajo

La Tabla 1 resume las tareas realizadas, sus responsables, plazos y relación con los objetivos del proyecto. Todas las actividades consideradas para esta fase se encuentran completadas.

Tabla 1. Plan de trabajo

ID	Tarea y entregable	Responsable	Plazo	Estado	Relación
1	Definir problemática, alcance y objetivos.	Isabel	30/06	Completado	O1; RF-01 y RF-02
2	Revisar literatura reciente y requisitos.	Gabriel	07/07	Completado	O1 y O4; RNF
3	EDA, limpieza y preprocesamiento 70/15/15.	Gabriel	15/07	Completado	O1; RF-03 a RF-05
4	Entrenar regresión logística, RF y XGBoost.	Isabel	23/07	Completado	O2; RF-06 y RF-07
5	Comparar pesos, ROS y SMOTE.	Isabel	23/07	Completado	O2 y O3; RF-07
6	Entrenar tres arquitecturas de DL.	Isabel	24/07	Completado	O2 y O3; RF-06
7	Seleccionar XGBoost + ROS y evaluar en prueba.	Isabel	24/07	Completado	O3; RF-07 y RNF confiabilidad
8	Refinar umbral e hiperparámetros usando validación.	Isabel	03/08	Completado	O3; selección de arquitectura
9	Crear notebook SHAP global/local, explicar tres casos y redactar la sección 5.3.	Diego	09/08	Completado	O4; RF-10 y explicabilidad
10	Implementar backend de inferencia y validación.	Gabriel	12/08	Completado	O4; RF-08 y RF-09
11	Crear interfaz Streamlit y monitoreo básico.	Gabriel	16/08	Completado	O4; usabilidad y monitoreabilidad
12	Actualizar README, requirements y probar la ejecución completa.	Diego	20/08	Completado	RF-12; reproducibilidad
13	Integrar refinamiento, SHAP, aplicación y resultados al informe.	Diego	23/08	Completado	O3 y O4
14	Revisión cruzada, APA, formato y entrega PDF.	Diego	24-28/08	Completado	Todos

IV. Desarrollo del proyecto

4.1 Análisis exploratorio de datos

4.1.1 Objetivo y procedencia del conjunto de datos

El proyecto busca predecir el abandono de clientes (churn) a partir de información demográfica, contractual, de uso, facturación, soporte y satisfacción. Se utilizó el conjunto Customer Churn Prediction Business Dataset, publicado en Kaggle, compuesto por 10.000 observaciones y 32 variables. Cada fila representa un cliente y la variable objetivo churn toma el valor 0 cuando el cliente permanece y 1 cuando abandona (Miah, 2025).

La elección del conjunto de datos es pertinente porque combina variables numéricas y categóricas, presenta una clase minoritaria de interés comercial y permite abordar el ciclo completo de un proyecto de clasificación: control de calidad, análisis exploratorio, preprocesamiento, balanceo, comparación de modelos y evaluación.

Cabe señalar que el conjunto de datos probablemente es sintético: la propia página del dataset en Kaggle lo describe como «synthetic but business-realistic», generado mediante reglas de negocio combinadas con ruido probabilístico para simular correlaciones plausibles entre variables (Miah, 2025). Esto no invalida su uso con fines académicos, pero implica que las métricas obtenidas a lo largo de este informe deben interpretarse como una evaluación preliminar del enfoque metodológico, y no como una medición del desempeño esperado sobre datos reales de clientes.

4.1.2 Descripción de las variables

Tabla 2. Diccionario completo de variables del dataset

Variable	Tipo	Descripción	Rango o categorías
customer_id	Catórica	Identificador único del cliente.	10000 categorías
gender	Catórica	Género declarado del cliente.	Female, Male
age	Núérica	Edad del cliente en años.	18 a 74
country	Catórica	País asociado al cliente.	Australia, Bangladesh, Canada, Germany, India, UK, USA
city	Catórica	Ciudad asociada al cliente.	Berlin, Delhi, Dhaka, London, New York, Sydney, Toronto
customer_segment	Catórica	Segmento comercial del cliente.	Enterprise, Individual, SME
tenure_months	Núérica	Antigüedad del cliente en meses.	1 a 59
signup_channel	Catórica	Canal utilizado para registrarse.	Mobile, Referral, Web
contract_type	Catórica	Tipo o duración del contrato.	Monthly, Quarterly, Yearly
monthly_logins	Núérica	Cantidad de inicios de sesión mensuales.	0 a 54
weekly_active_days	Núérica	Días activos por semana.	0 a 7
avg_session_time	Núérica	Duración promedio de sesión.	1 a 41.9964
features_used	Núérica	Cantidad de funcionalidades utilizadas.	1 a 15
usage_growth_rate	Núérica	Variación proporcional del uso.	-0.58 a 0.54
last_login_days_ago	Núérica	Días transcurridos desde el último acceso.	0 a 80
monthly_fee	Núérica	Tarifa mensual pagada por el cliente.	10 a 100
total_revenue	Núérica	Ingreso total acumulado por el cliente.	10 a 5900
payment_method	Catórica	Método de pago principal.	Bank Transfer, Card, PayPal
payment_failures	Núérica	Cantidad de fallos de pago.	0 a 5
discount_applied	Catórica	Indica si se aplicó un descuento.	No, Yes
price_increase_last_3m	Catórica	Indica si hubo aumento de precio en los últimos tres meses.	No, Yes
support_tickets	Núérica	Cantidad de tickets de soporte.	0 a 7
avg_resolution_time	Núérica	Tiempo promedio de resolución de tickets.	1 a 61.8219
complaint_type	Catórica	Tipo de reclamo o ausencia de reclamo.	Billing, No complaint, Service, Technical
csat_score	Núérica	Puntaje de satisfacción del cliente (CSAT).	1 a 5
escalations	Núérica	Cantidad de escalaciones de soporte.	0 a 4
email_open_rate	Núérica	Proporción de correos abiertos.	0.1 a 0.9
marketing_click_rate	Núérica	Proporción de clics en campañas.	0.01 a 0.5
nps_score	Núérica	Net Promoter Score.	-100 a 100
survey_response	Catórica	Respuesta cualitativa de encuesta.	Neutral, Satisfied, Unsatisfied
referral_count	Núérica	Cantidad de referidos.	0 a 7
churn	Binaria objetivo	Variable objetivo: 1 si abandona y 0 si permanece.	0 = permanece; 1 = abandona

Nota. La clasificación corresponde al tipo conceptual de la variable. Las variables catóricas se codifican posteriormente mediante OneHotEncoder.

4.1.3 Evaluación de calidad, completitud y correctitud

La carga inicial se realizó con pandas. Se detectó que el texto None de complaint_type era interpretado automáticamente como valor ausente. Para evitar esta lectura incorrecta se utilizó keep_default_na=False y se consideraron faltantes únicamente las celdas realmente vacías. Luego, la categoría None fue normalizada a No complaint, conservando su significado de ausencia de reclamo.

- Completitud: 100 %; no se detectaron valores nulos reales en las 320.000 celdas.
- Duplicados: 0 filas completamente repetidas y 0 identificadores customer_id duplicados.
- Consistencia catórica: no se observaron espacios residuales ni categorías equivalentes escritas de manera distinta después de la normalización.
- Correctitud núérica: las edades, tasas, puntajes, tiempos, conteos e ingresos se encontraron dentro de rangos semánticamente plausibles.
- Trazabilidad: se conservaron los datos originales en data/raw y la versión depurada en data/processed.

4.1.4 Valores atípicos y decisiones de limpieza

Tabla 3. Variables con valores atípicos según el criterio IQR

Variable	Mínimo	Máximo	Atípicos	%
monthly_fee	10.00	100.00	513	5.13 %
total_revenue	10.00	5900.00	513	5.13 %
last_login_days_ago	0.00	80.00	471	4.71 %
csat_score	1.00	5.00	221	2.21 %
payment_failures	0.00	5.00	147	1.47 %
features_used	1.00	15.00	121	1.21 %
usage_growth_rate	-0.58	0.54	78	0.78 %
monthly_logins	0.00	54.00	49	0.49 %
escalations	0.00	4.00	43	0.43 %
avg_resolution_time	1.00	61.82	38	0.38 %
nps_score	-100.00	100.00	36	0.36 %
avg_session_time	1.00	42.00	27	0.27 %
support_tickets	0.00	7.00	18	0.18 %
referral_count	0.00	7.00	8	0.08 %

Nota. Los atípicos fueron identificados con los límites $Q1 - 1,5 \cdot IQR$ y $Q3 + 1,5 \cdot IQR$. No se eliminaron automáticamente, porque correspondían a valores posibles dentro del dominio.

Los valores atípicos se conservaron. Por ejemplo, una tarifa mensual de 100, un NPS de -100 o 80 días desde el último acceso son extremos estadísticos, pero no errores lógicos. Eliminarlos habría reducido información potencialmente predictiva. La decisión fue documentar su presencia y permitir que los modelos robustos a relaciones no lineales evalúen su utilidad.

4.1.5 Preparación para el modelamiento

La variable `customer_id` se excluyó por ser un identificador sin significado predictivo, mientras que `churn` se separó como variable objetivo. Las 30 variables restantes conformaron `X` y la etiqueta binaria conformó `y`. Se utilizó una partición estratificada de 70 % para entrenamiento (7.000 casos), 15 % para validación (1.500) y 15 % para prueba (1.500), manteniendo la proporción original de `churn` en los tres subconjuntos.

Las variables numéricas fueron estandarizadas con `StandardScaler` y las categóricas se transformaron con `OneHotEncoder(handle_unknown="ignore")`. El preprocesador se ajustó exclusivamente con entrenamiento y luego se aplicó a validación y prueba para evitar filtración de información. Las 30 variables predictoras originales se transformaron en 58 características y no se generaron valores nulos ni infinitos.

Dado el desbalance de 8,79 a 1, se calcularon pesos de clase de 0,557 para la clase 0 y 4,895 para la clase 1. Las técnicas de balanceo se aplicaron únicamente al conjunto de entrenamiento; validación y prueba conservaron su distribución real. El desbalance de clases es un desafío ampliamente documentado en problemas de clasificación (He & Garcia, 2009) y, específicamente, en la predicción de abandono de clientes (Burez & Van den Poel, 2009).

4.1.6 Análisis exploratorio y patrones estadísticos

a. Distribución de la variable objetivo

La variable objetivo presenta 8.979 clientes que permanecen (89,79 %) y 1.021 que abandonan (10,21 %). Esta distribución evidencia un desbalance importante. Un clasificador que predijera siempre la clase mayoritaria obtendría 89,79 % de exactitud, pero detectaría cero abandonos; por ello, la exactitud no puede ser la única medida de desempeño.

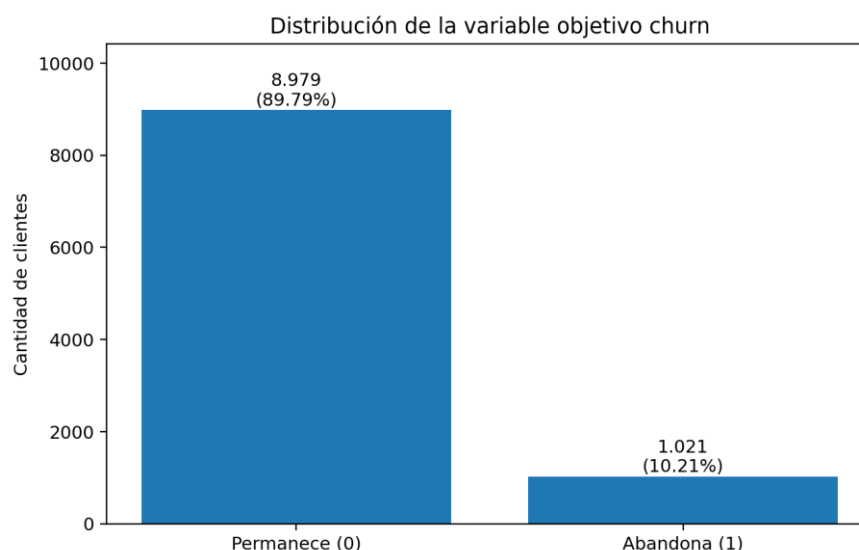


Figura 1. Distribución de la variable objetivo churn.

Fuente: elaboración propia.

b. Patrones en variables numéricas

Se compararon media y mediana por clase y se calculó el tamaño del efecto de Cohen. El mayor contraste se observó en `csat_score`: los clientes que permanecen registraron una media de 3,539 y quienes abandonan 3,028, con $|d| = 0,528$, magnitud moderada. Las diferencias en `tenure_months`, `payment_failures`, `monthly_logins` y `total_revenue` fueron pequeñas, pero consistentes con el fenómeno de abandono.

Tabla 4. Principales diferencias numéricas entre clientes que permanecen y abandonan

Variable	Media 0	Media 1	d de Cohen	Magnitud
<code>csat_score</code>	3.539	3.028	-0.528	Moderado
<code>tenure_months</code>	30.830	24.222	-0.389	Pequeño
<code>payment_failures</code>	0.474	0.738	0.374	Pequeño
<code>monthly_logins</code>	19.998	16.803	-0.326	Pequeño
<code>total_revenue</code>	1081.138	844.897	-0.232	Pequeño
<code>last_login_days_ago</code>	9.382	10.588	0.123	Muy pequeño

Nota. El signo indica la dirección y el valor absoluto indica la magnitud. La clase 0 permanece y la clase 1 abandona.

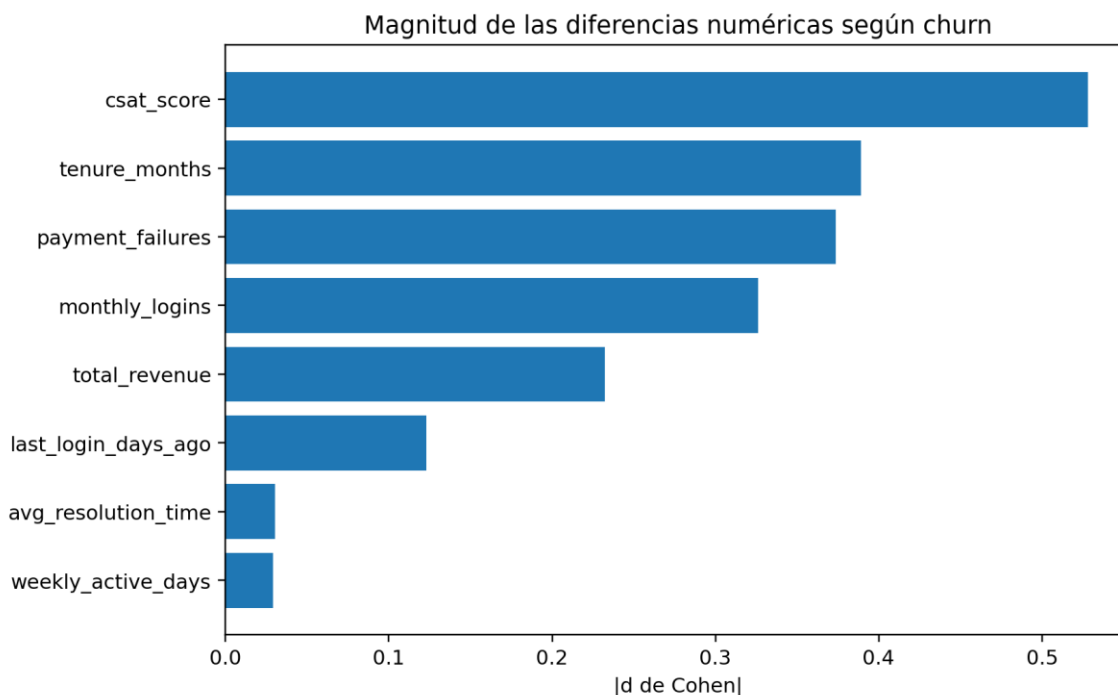
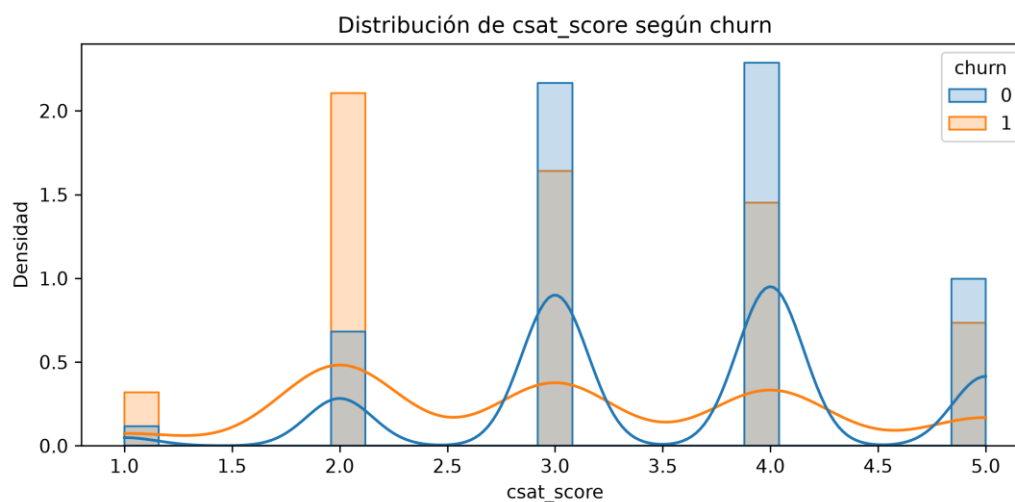


Figura 2. Magnitud de las diferencias numéricas según churn.

Fuente: elaboración propia.

En términos de negocio, los clientes que abandonan presentan menor satisfacción, menor antigüedad, menos accesos mensuales, más fallos de pago y menor ingreso acumulado. Sin embargo, existe una superposición considerable entre las distribuciones, por lo que ninguna variable numérica permite separar las clases de manera individual.



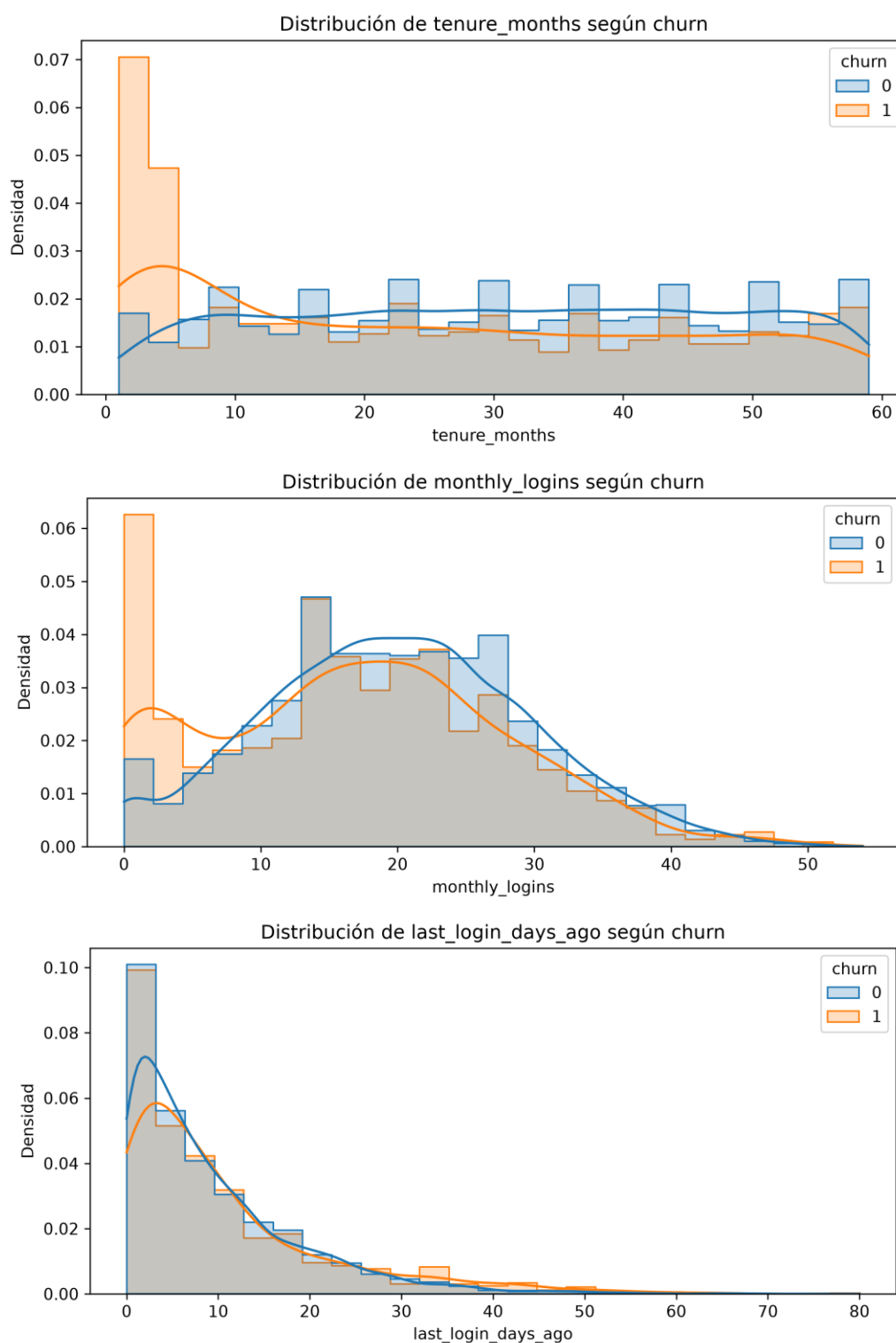


Figura 3. Histogramas de *csat_score*, *tenure_months*, *monthly_logins* y *last_login_days_ago* según churn.
Fuente: elaboración propia.

Los histogramas confirman que las distribuciones de clientes que permanecen y abandonan presentan una superposición importante, sin que ninguna variable numérica separe completamente a ambos grupos. No obstante, se observan desplazamientos visibles en `csat_score`, `tenure_months` y `monthly_logins` hacia valores menores entre quienes abandonan, consistentes con las diferencias de medias reportadas en la Tabla 4. Estos desplazamientos son señales útiles, pero solo alcanzan valor predictivo al combinarse con otras características dentro de un modelo multivariable.

c. Patrones en variables categóricas

Las mayores brechas de tasa de churn se observaron en `customer_segment`, `survey_response`, `signup_channel` y `discount_applied`. Los clientes SME registraron 10,9 % de churn frente a 9,4 % en Enterprise. Los clientes insatisfechos alcanzaron 11,1 %, mientras que la respuesta neutral presentó 9,7 %. Los clientes sin descuento registraron 10,5 % frente a 9,6 % entre quienes recibieron descuento.

Variables categóricas con mayor brecha de churn

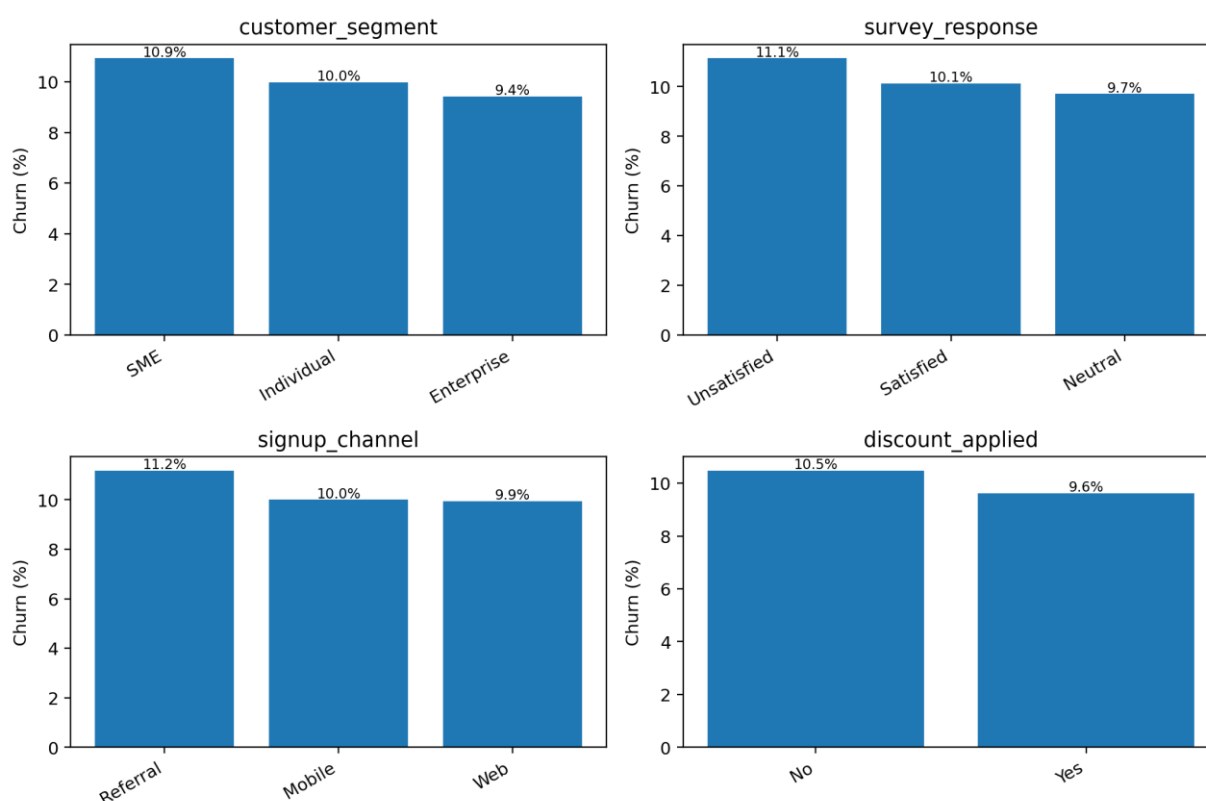


Figura 4. Variables categóricas con mayor brecha de churn.

Fuente: elaboración propia.

Aunque estas fueron las mayores diferencias categóricas, ninguna superó aproximadamente 1,5 puntos porcentuales. Por consiguiente, las categorías aisladas poseen una capacidad explicativa limitada y su valor principal puede surgir al combinarse con otras características.

d. Correlaciones y relaciones conjuntas

Las correlaciones lineales con churn fueron débiles. Los valores de mayor magnitud fueron csat_score (-0,158), tenure_months (-0,117), payment_failures (0,112), monthly_logins (-0,098), total_revenue (-0,070) y last_login_days_ago (0,037). Entre predictores, total_revenue se correlacionó con monthly_fee (0,71) y tenure_months (0,59), lo que evidencia redundancia parcial.

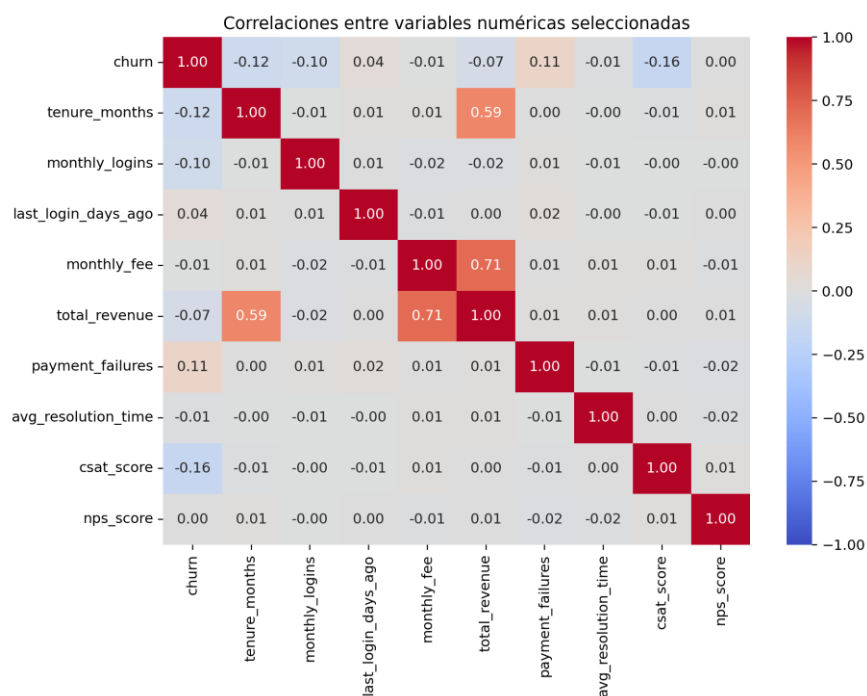


Figura 5. Mapa de correlaciones entre variables numéricas seleccionadas.

Fuente: elaboración propia.

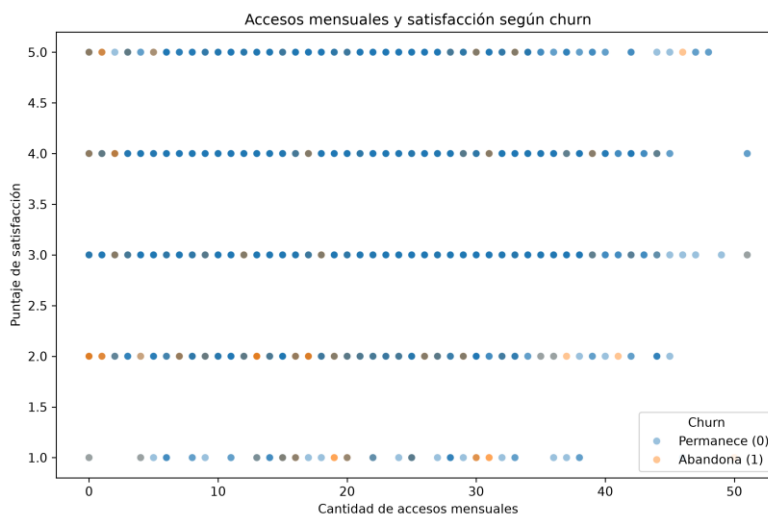


Figura 6. Relación conjunta entre accesos mensuales y satisfacción según churn.

Fuente: elaboración propia.

El gráfico de dispersión, construido sobre una muestra aleatoria reproducible de 3.000 registros, no muestra una frontera clara entre clientes que permanecen y abandonan al combinar accesos mensuales y satisfacción. Los casos de abandono aparecen con mayor frecuencia relativa en niveles bajos de ambas variables, aunque también existen clientes que permanecen dentro de esos mismos rangos. Las franjas horizontales se explican porque `csat_score` es una variable discreta entre 1 y 5. En conjunto, estas dos variables aportan señal relacionada con el churn, pero no permiten predecirlo de forma aislada.

Los resultados del análisis exploratorio indican que el churn es multivariable. No existe una característica dominante: la señal está distribuida entre satisfacción, antigüedad, actividad, pagos y atributos comerciales. Este patrón justifica comparar modelos lineales y no lineales capaces de combinar señales débiles.

4.2 Selección y justificación de técnicas

4.2.1 Aprendizaje supervisado y no supervisado

El problema es supervisado porque el dataset incluye una etiqueta histórica de churn. El objetivo es aprender una función que asigne una probabilidad de abandono a cada cliente y permita priorizar acciones de retención. Las técnicas no supervisadas, como K-means, podrían complementar el análisis mediante segmentación, pero no optimizan directamente la detección de churn; por ello se dejan como extensión futura.

4.2.2 Modelos de machine learning seleccionados

- Regresión logística: línea base interpretable y eficiente para estimar probabilidades en una variable binaria.
- Random Forest: ensamble capaz de capturar relaciones no lineales e interacciones, robusto frente a escalas y valores extremos (Breiman, 2001).
- XGBoost: boosting de árboles con regularización, competitivo en datos tabulares y con buen desempeño reportado en churn (Chen & Guestrin, 2016; Ahmad et al., 2019).

4.2.3 Arquitecturas de deep learning seleccionadas

- MLP superficial (58-64-1): una capa oculta de 64 neuronas, activación ReLU y dropout 0,3.
- MLP profunda (58-128-64-32-1): tres capas ocultas, BatchNorm, ReLU y dropout 0,3.
- MLP residual: proyección a 64 dimensiones y dos bloques residuales con conexiones de salto.

No se seleccionaron CNN ni RNN porque el dataset no posee estructura espacial ni secuencial. La MLP residual permite contrastar una arquitectura más compleja para datos tabulares, mientras que TabNet queda como alternativa futura por su atención secuencial e interpretabilidad (Arik & Pfister, 2021; Gorishniy et al., 2021).

4.2.4 Métricas de evaluación

- Accuracy: proporción total de predicciones correctas; se reporta, pero no es suficiente por el desbalance.
- Precisión: proporción de alertas de churn que corresponden a abandonos reales.
- Recall: proporción de abandonos reales detectados; es prioritaria para el objetivo de retención.
- F1-score: media armónica entre precisión y recall.
- ROC-AUC: capacidad global de discriminar ambas clases a distintos umbrales.
- PR-AUC (Average Precision): resume la relación entre precisión y recall y es informativa cuando la clase positiva es minoritaria.
- Matriz de confusión: presenta verdaderos y falsos positivos y negativos.

Tabla 5. Trabajos previos y aporte a las decisiones del proyecto

Trabajo	Aporte principal	Relación con el proyecto
Vafeiadis et al. (2015)	Comparación de clasificadores y boosting para churn.	Sustenta la comparación multimodelo y las métricas.
Ahmad et al. (2019)	Evaluación de árboles, RF, GBM y XGBoost.	Confirma la pertinencia de XGBoost para churn.
Burez y Van den Poel (2009)	Análisis del desequilibrio en seis datasets de churn.	Justifica balanceo y métricas distintas de accuracy.
Chen y Guestrin (2016)	Presentación de XGBoost y sus optimizaciones.	Fundamenta el boosting seleccionado.
Chawla et al. (2002)	Propuesta de SMOTE.	Sustenta una estrategia de balanceo evaluada.
Gorishniy et al. (2021)	Comparación de modelos profundos tabulares.	Sustenta la MLP residual y ML frente a DL.
Arik y Pfister (2021)	Propuesta de TabNet.	Apoya la discusión de DL tabular interpretable.
Borisov et al. (2024)	Revisión de redes profundas para datos tabulares.	Contextualiza sus ventajas y limitaciones.

4.3 Comparación de técnicas

Tabla 6. Comparación conceptual de modelos de machine learning

Técnica	Complejidad	Interpretabilidad	Fortaleza	Limitación
Regresión logística	Baja	Alta	Probabilidades y referencia lineal.	Puede subajustar interacciones complejas.
Random Forest	Media	Media	No linealidad, robustez e interacciones.	Mayor costo y menor transparencia que LR.
XGBoost	Media-alta	Media	Rendimiento tabular y regularización.	Requiere ajuste y explicabilidad adicional.

Tabla 7. Comparación conceptual de arquitecturas de deep learning

Arquitectura	Estructura	Ventaja esperada	Riesgo o limitación
MLP superficial	58-64-1	Bajo costo; referencia neuronal.	Capacidad limitada.
MLP profunda	58-128-64-32-1	Interacciones jerárquicas.	Más parámetros y riesgo de sobreajuste.
MLP residual	64 + 2 bloques residuales	Mejor flujo del gradiente.	Mayor complejidad sin ventaja garantizada.

El dataset posee 10.000 registros y 58 características después del preprocesamiento, por lo que todos los candidatos pudieron entrenarse en CPU. Se usaron configuraciones moderadas y semillas fijas, sin una búsqueda exhaustiva de hiperparámetros. Esta restricción favorece la reproducibilidad y evita complejidad innecesaria para un prototipo académico.

4.4 Requisitos del proyecto

Los requisitos se vinculan con el alcance del prototipo. El estado indica qué componentes ya están respaldados por notebooks y artefactos del repositorio y cuáles deben completarse durante la integración.

Tabla 8. Requisitos funcionales y estado de avance

Código	Requisito	Relación con el alcance	Estado
RF-01	Cargar el dataset CSV.	Base del análisis histórico.	Completado
RF-02	Identificar churn como variable objetivo.	Clasificación binaria.	Completado
RF-03	Limpiar nulos, duplicados e inconsistencias.	Calidad de datos.	Completado
RF-04	Codificar categorías y escalar variables.	Datos mixtos.	Completado
RF-05	Dividir train, validación y prueba.	Evaluación sin filtración.	Completado
RF-06	Entrenar 3 modelos ML y 3 arquitecturas DL.	Comparación técnica.	Completado
RF-07	Calcular métricas y matrices de confusión.	Selección objetiva.	Completado
RF-08	Ingresar datos de un cliente en una interfaz.	Uso del prototipo.	Completado
RF-09	Entregar probabilidad y nivel de riesgo.	Apoyo a retención.	Completado
RF-10	Explicar predicciones mediante SHAP.	Transparencia.	Completado
RF-11	Registrar métricas y predicciones agregadas.	Trazabilidad y monitoreo.	Completado
RF-12	Mantener código e instrucciones en GitHub.	Colaboración y reproducibilidad.	Completado

Tabla 9. Requisitos no funcionales

Categoría	Requisito	Justificación	Estado
Usabilidad	Interfaz clara para usuarios no técnicos.	Facilita decisiones de negocio.	Completado
Explicabilidad	Importancia global y explicación local con SHAP.	Justifica acciones de retención.	Completado
Escalabilidad	Separar preprocesamiento, inferencia e interfaz.	Permite incorporar registros y modelos.	Completado
Seguridad	Usar datos anonimizados y no guardar información sensible.	Reduce riesgos de privacidad.	Completado
Confiabilidad	Evaluar con validación y prueba separadas.	Reduce sobreajuste.	Completado
Monitoreabilidad	Registrar volumen, riesgo y distribución de predicciones.	Detecta cambios de comportamiento.	Completado
Mantenibilidad	Organizar código, notebooks y artefactos.	Facilita continuidad.	Completado
Reproducibilidad	Mantener requirements y pasos de ejecución.	Permite replicar resultados.	Completado
Rendimiento	Responder en pocos segundos en CPU.	Uso fluido del prototipo.	Completado
Ética	Apoyar, no automatizar, decisiones sobre clientes.	Evita decisiones injustas.	Completado

4.5 Selección de la arquitectura y del modelo

4.5.1 Entradas, salida y configuraciones

Las 30 variables predictoras originales se transformaron en 58 características. Los modelos reciben ese vector y entregan una probabilidad de churn; con umbral 0,5 se obtiene la clase final. La regresión logística se entrenó con 2.000 iteraciones; Random Forest con 250 árboles, profundidad máxima 8 y mínimo de 4 registros por hoja; XGBoost con 120 estimadores, profundidad 3, learning rate 0,05, subsample 0,8 y colsample_bytree 0,8.

Las redes utilizaron AdamW, lotes de 256, BCEWithLogitsLoss ponderada y un máximo de 30 épocas. La MLP superficial completó el presupuesto, mientras que la profunda y la residual detuvieron su entrenamiento en las épocas 17 y 13 por ausencia de mejora del ROC-AUC durante cinco épocas consecutivas. En todos los casos se recuperaron los pesos de la mejor época.

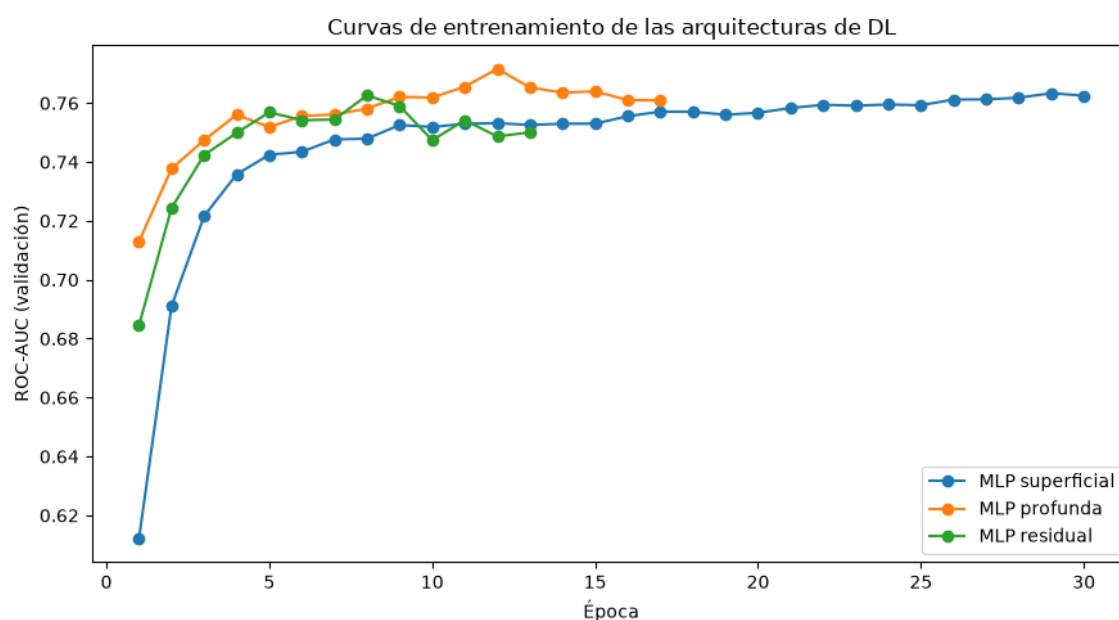


Figura 7. Curvas de entrenamiento de las arquitecturas de deep learning.

Fuente: elaboración propia.

4.5.2 Selección inicial

Random Forest + ROS obtuvo el mayor F1 de validación (0,406), mientras que XGBoost + ROS alcanzó el mayor recall (0,843), ROC-AUC (0,826) y PR-AUC (0,333). La diferencia de F1 entre ambos fue 0,010. Debido a que el proyecto prioriza detectar clientes en riesgo, se seleccionó XGBoost + ROS como modelo final, aceptando más falsas alarmas a cambio de omitir menos abandonos.

4.5.3 Refinamiento posterior

Se buscaron hiperparámetros mediante RandomizedSearchCV (35 combinaciones) sobre validación cruzada estratificada de 5 pliegues, aplicada exclusivamente dentro del conjunto de entrenamiento; el conjunto de validación no participó en esta etapa. Se usó un pipeline de imbalanced-learn que aplica RandomOverSampler solo al pliegue

de entrenamiento de cada fold, optimizando PR-AUC en vez de F1 por ser independiente del umbral de decisión. El espacio de búsqueda incluyó profundidad máxima, tasa de aprendizaje, número de árboles, submuestreo, columnas por árbol y tres parámetros de regularización (gamma, alpha y lambda).

La configuración ganadora fue más conservadora que la original: profundidad máxima 2 (antes 3), tasa de aprendizaje 0,02 (antes 0,05), 250 árboles (antes 120), submuestreo 0,7 y columnas por árbol 0,7.

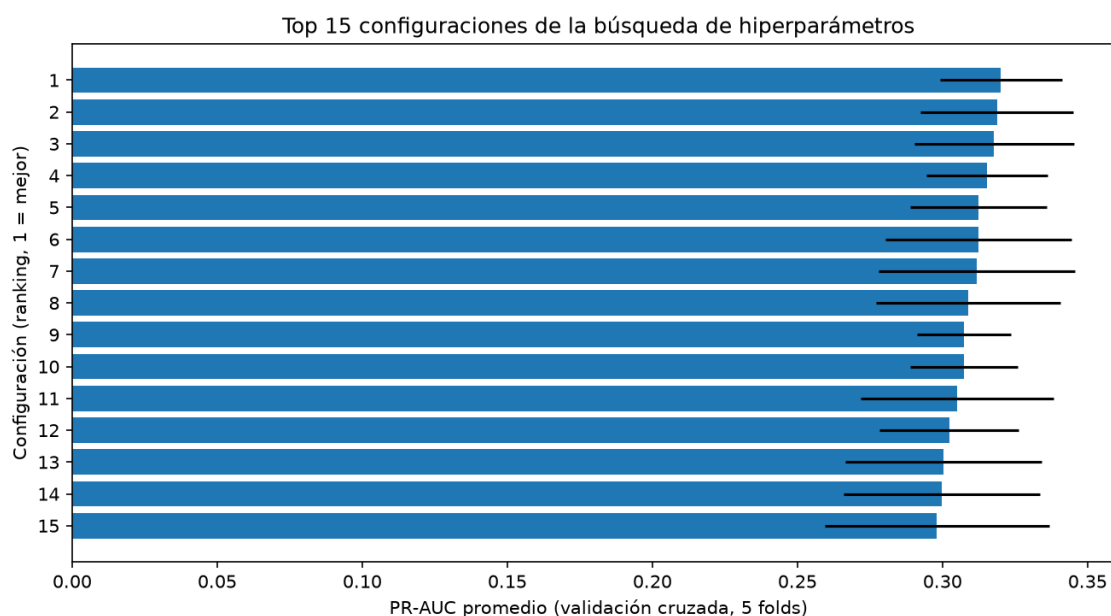


Figura 8. Búsqueda de hiperparámetros de XGBoost (validación cruzada, top 15 configuraciones).

Fuente: elaboración propia.

Con los hiperparámetros fijos, se ajustó el umbral de decisión sobre validación. El primer criterio probado — maximizar F1— llevó a un umbral de 0,66, pero ese umbral bajaba el recall de validación de 0,843 a 0,569: una caída incompatible con la prioridad de negocio del proyecto, que es detectar la mayor cantidad posible de clientes en riesgo de abandono, y suficiente por sí sola para descartarlo. Durante el desarrollo se obtuvo además una lectura exploratoria sobre el conjunto de prueba con ese umbral (recall 0,44; ver sección 5.2), pero esta no se utilizó para seleccionar el umbral definitivo: la decisión de corregir el criterio se basó únicamente en la evidencia de validación. El umbral definitivo es el que mantiene recall $\geq 0,80$ en validación, maximizando precision dentro de esa restricción. Ese umbral resultó ser 0,56. A partir de los mismos cortes se definieron tres bandas de riesgo (Bajo, Medio, Alto) que utilizará el prototipo para mostrar el resultado.

La Tabla 10 compara el efecto de cada etapa del refinamiento en validación.

Tabla 10. Efecto del refinamiento de XGBoost + ROS en validación.

Configuración	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Baseline (umbral 0,50)	0,737	0,259	0,843	0,396	0,826	0,333
Refinado (umbral 0,50)	0,724	0,249	0,843	0,384	0,829	0,341
Refinado (umbral 0,56, recall $\geq 0,80$)	0,740	0,255	0,804	0,387	0,829	0,341

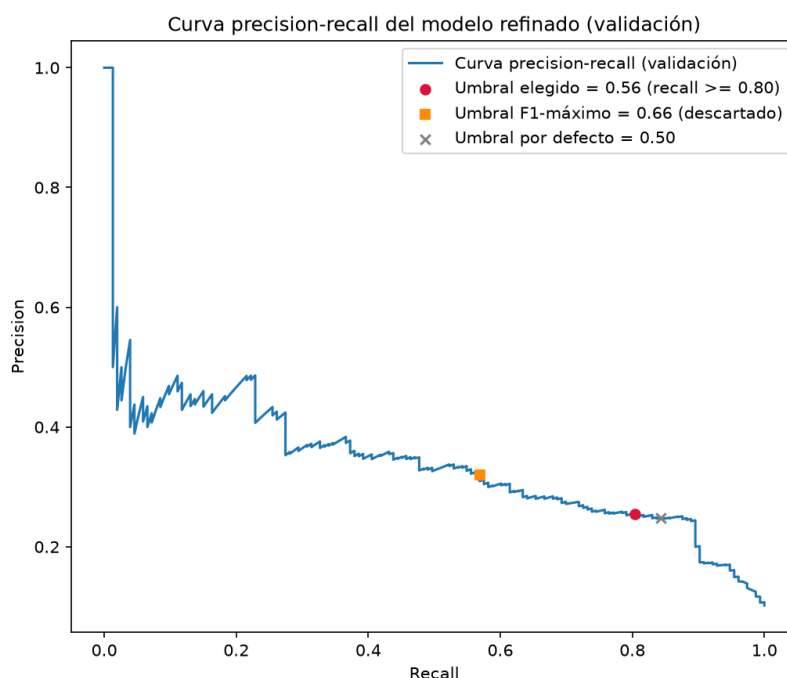


Figura 9. Curva precision-recall del modelo XGBoost refinado y umbrales evaluados en validación.

4.5.4 Refinamiento de la arquitectura de red neuronal

Además del modelo final, se ensayaron variantes de la MLP profunda (la mejor red del notebook 05 en recall, ROC-AUC y PR-AUC) para demostrar el proceso de refinamiento de una arquitectura neuronal, sin reabrir la selección de XGBoost como modelo final. Se compararon tres configuraciones en validación, manteniendo el mismo esquema de entrenamiento (lotes de 256, máximo 30 épocas, paciencia 5): (A) la arquitectura original 58-128-64-32-1 con dropout 0,3; (B) una versión más angosta y regularizada (58-64-32-1, dropout 0,2, mayor weight decay y un programador de tasa de aprendizaje); (C) la arquitectura original con un pos_weight más suave (raíz cuadrada de la relación de desbalance) y detención temprana basada en PR-AUC en vez de ROC-AUC.

Tabla 11. Comparación de configuraciones de la MLP profunda en validación.

Configuración	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
A - Baseline	0,668	0,198	0,739	0,312	0,761	0,242
B - Regularizada	0,699	0,215	0,739	0,333	0,777	0,250
C - Orientada a precisión	0,823	0,260	0,399	0,314	0,762	0,244

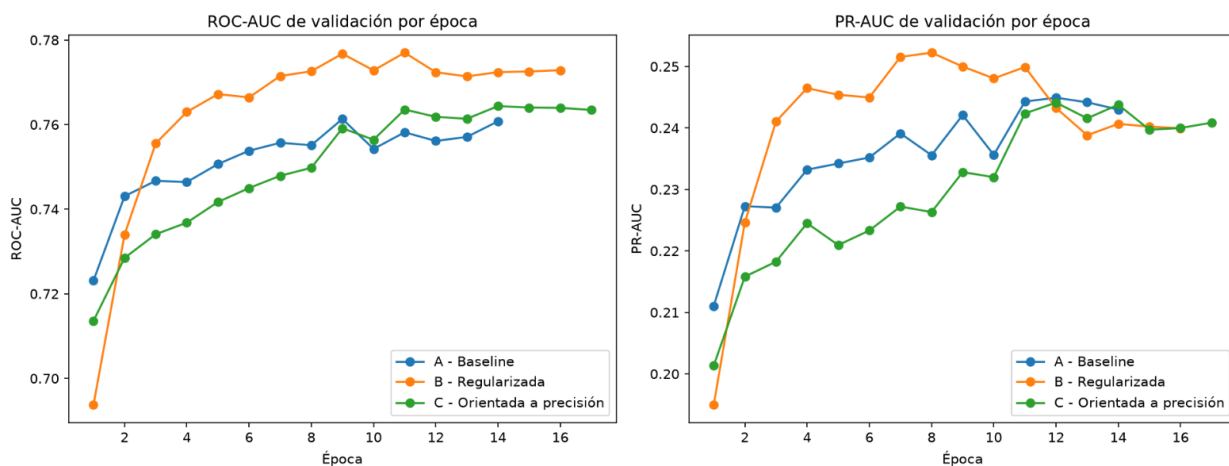


Figura 10. Evolución del ROC-AUC y PR-AUC de validación durante el entrenamiento de las configuraciones MLP.

Fuente: elaboración propia.

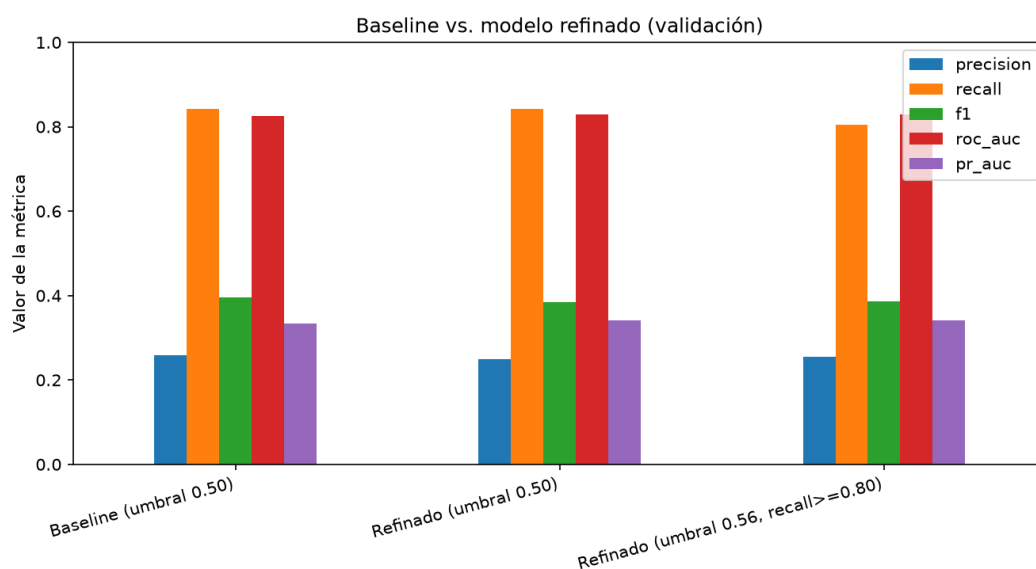


Figura 11. Comparación del baseline y las configuraciones refinadas de XGBoost en validación.

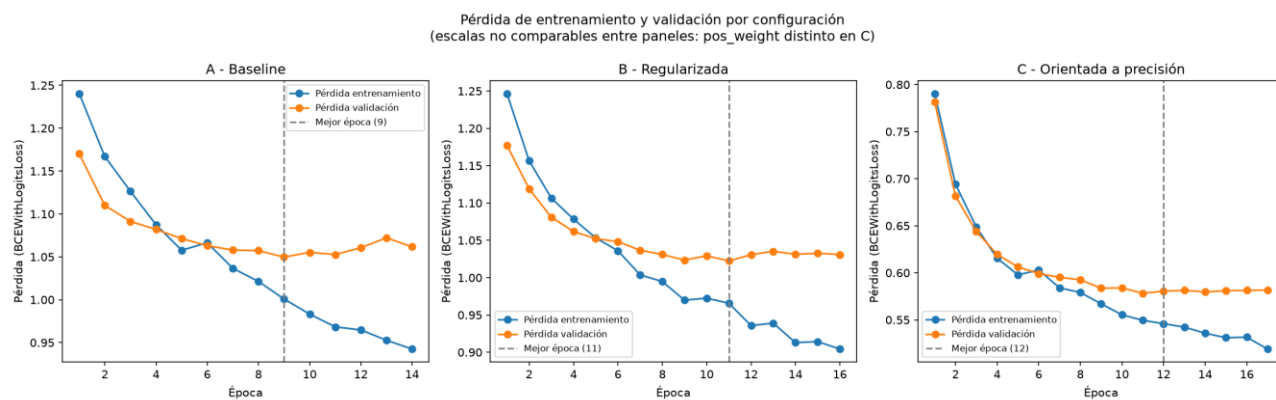


Figura 12. Pérdida de entrenamiento y validación por configuración (BCEWithLogitsLoss), con la época óptima marcada.

Fuente: elaboración propia.

Las tres configuraciones muestran una brecha pequeña entre la pérdida de entrenamiento y de validación en su época óptima (por ejemplo, B alcanza 0,97 de pérdida de entrenamiento frente a 1,02 de validación en la época 11), lo que indica una convergencia razonable sin sobreajuste severo. Los valores absolutos de la configuración C son notoriamente más bajos que los de A y B (alrededor de 0,55 frente a 1,0), pero esto no refleja un mejor ajuste: se debe a que C usa un `pos_weight` menor, que reduce la magnitud de la pérdida independientemente de la calidad de las predicciones. Por eso los paneles se presentan por separado y la comparación entre configuraciones debe hacerse con las métricas de la Tabla 11, no con los valores de pérdida.

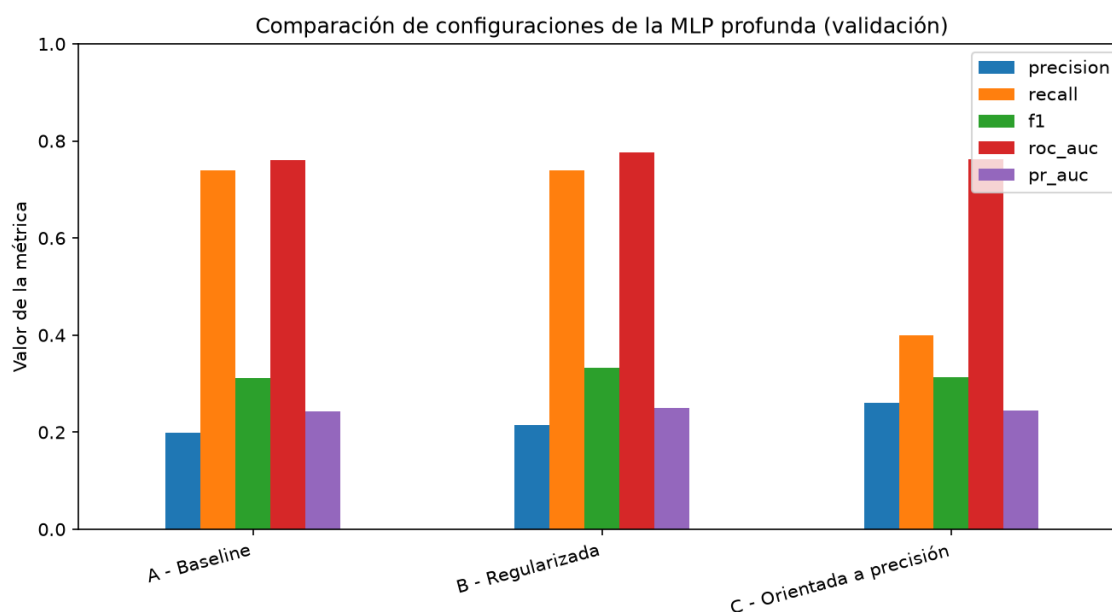


Figura 13. Comparación de las tres configuraciones de la MLP profunda en validación.

Fuente: elaboración propia.

La configuración B obtuvo el mejor F1 (0,333 frente a 0,312 de la original), mejorando además precisión, ROC-AUC y PR-AUC sin perder recall — lo que sugiere que una arquitectura más compacta y regularizada puede generalizar mejor en este conjunto de datos para las 7.000 filas de entrenamiento disponibles. La configuración C reprodujo, a menor escala, el mismo problema encontrado al ajustar el umbral de XGBoost en la sección 4.5.3: mejoró precisión y accuracy, pero hundió el recall de 0,739 a 0,399. Ninguna variante se acerca al desempeño de XGBoost + ROS (F1 0,387 en validación tras su propio refinamiento), por lo que la conclusión de la sección 4.5.2 se mantiene: XGBoost + ROS sigue siendo el modelo final del proyecto.

4.6 Elaboración de modelos y balanceo

4.6.1 Diseño experimental

Se usaron 7.000 casos para entrenamiento, 1.500 para validación y 1.500 para prueba, con partición estratificada 70/15/15. El preprocesador se ajustó exclusivamente con entrenamiento. Se fijó `random_state = 42` y semillas equivalentes en NumPy y PyTorch.

Para cada modelo de ML se compararon cuatro escenarios: sin balanceo, pesos de clase, RandomOverSampler (ROS) y SMOTE. ROS y SMOTE se aplicaron solo sobre entrenamiento y sin pesos adicionales, porque el remuestreo ya modificaba la distribución. Validación y prueba conservaron la proporción real de churn. SMOTE genera ejemplos sintéticos interpolando entre vecinos de la clase minoritaria (Chawla et al., 2002), a diferencia de ROS, que duplica ejemplos existentes.

Tabla 12. Efecto de las estrategias de balanceo por modelo en validación

Modelo	Escenario	Accuracy	Precisión	Recall	F1	ROC-AUC	PR-AUC
Regresión logística	Sin balanceo	0,896	0,286	0,013	0,025	0,751	0,223
Regresión logística	Pesos	0,700	0,207	0,686	0,318	0,750	0,224
Regresión logística	ROS	0,705	0,215	0,712	0,330	0,751	0,221
Regresión logística	SMOTE	0,702	0,207	0,680	0,318	0,738	0,219
Random Forest	Sin balanceo	0,898	0,000	0,000	0,000	0,818	0,327
Random Forest	Pesos	0,817	0,304	0,614	0,407	0,820	0,312
Random Forest	ROS	0,776	0,278	0,752	0,406	0,818	0,302
Random Forest	SMOTE	0,869	0,186	0,085	0,117	0,773	0,231
XGBoost	Sin balanceo	0,898	0,500	0,059	0,105	0,818	0,322
XGBoost	Pesos	0,731	0,248	0,810	0,380	0,819	0,329
XGBoost	ROS	0,737	0,259	0,843	0,396	0,826	0,333
XGBoost	SMOTE	0,895	0,368	0,046	0,081	0,793	0,263

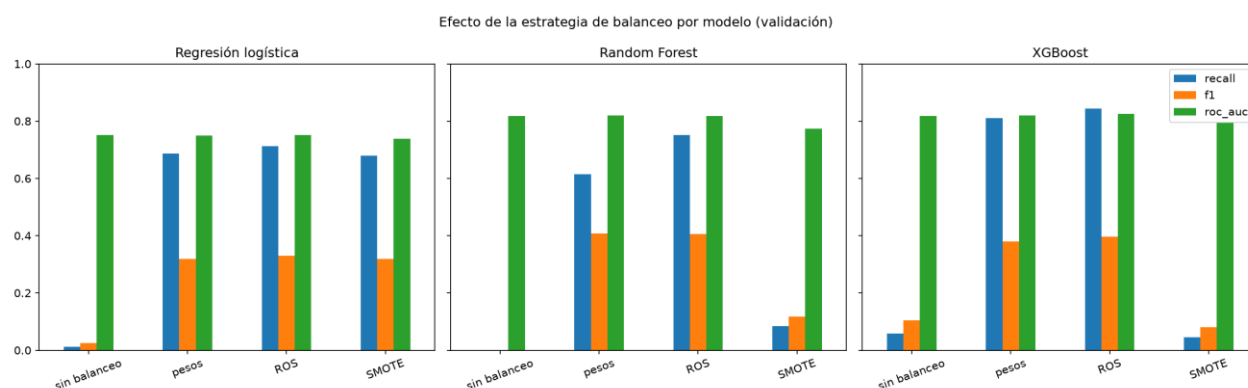


Figura 14. Efecto de las estrategias de balanceo sobre recall, F1 y ROC-AUC.

Fuente: elaboración propia.

Los modelos sin balanceo alcanzaron una accuracy alta, pero un recall muy bajo o nulo. Los pesos y ROS mejoraron la sensibilidad de forma consistente. SMOTE presentó resultados inferiores en Random Forest y XGBoost; una posible explicación es que la interpolación sobre variables one-hot genere observaciones sintéticas poco representativas, hipótesis que requiere análisis adicional.

ROS se seleccionó para los tres modelos. En Random Forest, los pesos obtuvieron F1 0,407 y ROS 0,406; al ser una diferencia marginal, se priorizó ROS por elevar el recall de 0,614 a 0,752.

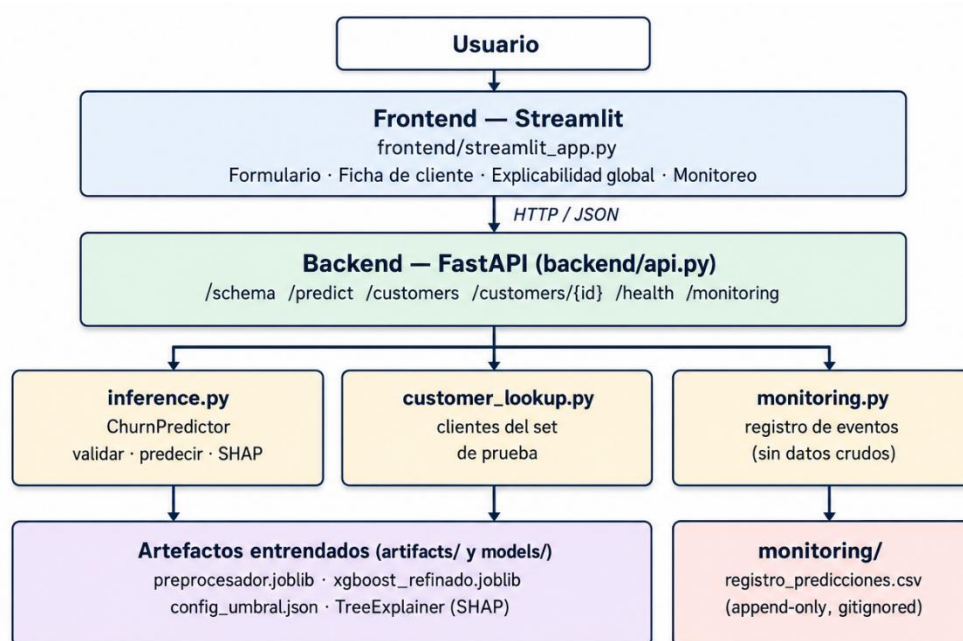
4.7 Desarrollo frontend y backend

4.7.1 Arquitectura del sistema

El prototipo utiliza una arquitectura por capas compuesta por un frontend en Streamlit y un backend en FastAPI, comunicados exclusivamente mediante HTTP (Figura 15). Esta separación permite modificar o escalar la interfaz y la lógica de predicción de manera independiente.

El backend se divide en módulos responsables de validar las 30 variables de entrada, cargar los artefactos entrenados, ejecutar las predicciones, consultar clientes del conjunto de prueba y registrar el monitoreo. La API expone seis rutas: `/schema`, `/health`, `/predict`, `/customers`, `/customers/{id}` y `/monitoring`.

Esta organización responde a los requisitos de escalabilidad y mantenibilidad definidos en la sección 4.4, ya que separa la interfaz, la inferencia y los artefactos del modelo, y permite probar cada componente de forma aislada.



Frontend y backend son procesos independientes que se comunican solo por HTTP: se pueden escalar, desplegar o reemplazar por separado sin tocar la lógica de predicción.

Figura 15. Arquitectura del sistema: separación entre frontend, backend, lógica de negocio y artefactos entrenados.

Fuente: elaboración propia.

4.7.2 Backend de inferencia

Al iniciar el servidor, la clase `ChurnPredictor` carga una sola vez el preprocesador, el modelo XGBoost, la configuración de umbrales y el explicador SHAP. De esta manera, los artefactos no se vuelven a cargar en cada solicitud.

El backend valida la presencia y el tipo de las variables, rechaza valores imposibles y categorías desconocidas, y advierte cuando un valor numérico se encuentra fuera del rango histórico. Posteriormente, aplica el mismo preprocesador utilizado durante el entrenamiento y calcula la probabilidad de churn mediante `predict_proba()`.

La probabilidad se clasifica en riesgo Bajo, Medio o Alto según la banda baja de 0,24 y el umbral de decisión de 0,56.

La consistencia de este proceso se verificó con 20 clientes del conjunto de prueba, obteniendo exactamente las mismas probabilidades que al utilizar directamente los artefactos desde los notebooks.

El backend cuenta con 43 pruebas automatizadas que cubren validaciones, predicciones, límites de riesgo, explicaciones SHAP, monitoreo, manejo de errores y los seis endpoints de la API. Las pruebas utilizan los artefactos reales y se ejecutan mediante pytest.

4.7.3 Frontend

La interfaz desarrollada en Streamlit contiene cuatro pestañas: Predicción individual, Ficha de cliente, Explicabilidad global y Monitoreo.

En Predicción individual, el usuario completa las 30 variables agrupadas por temática, con valores iniciales obtenidos de la mediana o la moda del conjunto de datos (Figura 16). En Ficha de cliente, puede seleccionar un cliente real del conjunto de prueba.

En ambos casos, el resultado se presenta en una ventana modal que incluye la probabilidad de churn, la banda de riesgo, la comparación con el umbral, los principales factores explicativos y el gráfico SHAP correspondiente (Figura 17). Al cerrar la ventana, los datos ingresados se conservan.

Backend conectado — modelo: xgboost_refinado.joblib

Predicción individual | Ficha de cliente | Explicabilidad global | Monitoreo

Ingresar un cliente nuevo

Demografía

Género: Masculino | Edad: 46 | País: Bangladesh | Ciudad: Londres

Contrato y uso

Segmento de cliente: Individual | Canal de registro: Sitio web | Tipo de contrato: Mensual

Antigüedad (meses): 30 | Inicios de sesión mensuales: 20 | Días activos por semana: 3

Duración promedio de sesión (min): 15.16 | Funciones utilizadas: 5 | Crecimiento de uso reciente: 0.02

Días desde el último acceso: 6

Facturación

Figura 16. Formulario de ingreso de datos de un cliente nuevo, agrupado por secciones.

Fuente: elaboración propia.

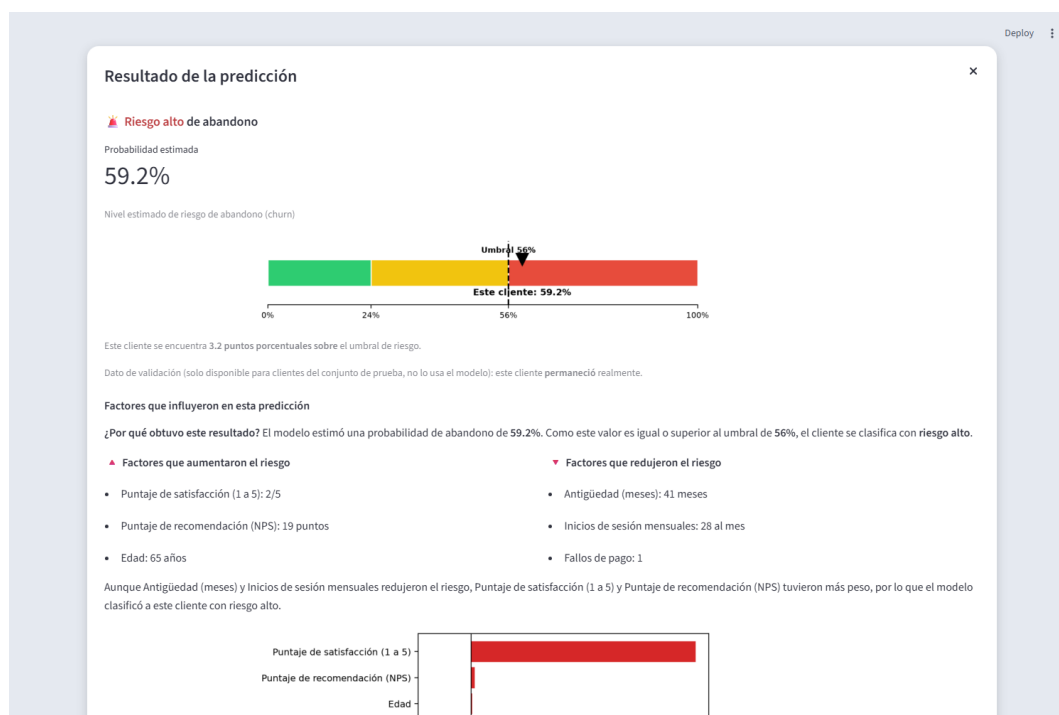


Figura 17. Resultado de la predicción en la ventana modal, con la barra de probabilidad y los factores por dirección.

Fuente: elaboración propia.

4.7.4 Explicabilidad y monitoreo

Para facilitar la interpretación de los resultados, la aplicación complementa el gráfico técnico de SHAP con una explicación en lenguaje simple. Esta presenta por separado hasta tres factores que aumentan y tres que reducen el riesgo, incluyendo el valor real de cada variable. En las variables categóricas, las contribuciones generadas por el one-hot encoding se agregan antes de mostrarse al usuario.

La explicación también indica que los factores corresponden a asociaciones aprendidas por el modelo y no necesariamente a relaciones causales. La pestaña Explicabilidad global muestra la importancia general de las variables calculada previamente en el notebook de SHAP (Figura 18).

Cada predicción registra fecha, origen, probabilidad, banda de riesgo, umbral y tiempo de respuesta, sin almacenar las variables originales del cliente. La pestaña Monitoreo muestra el volumen de predicciones, la latencia promedio, la proporción de clientes en riesgo Alto y la distribución de resultados (Figura 19).

Las métricas obtenidas en el conjunto de prueba se presentan por separado, ya que todavía no se dispone del churn real posterior de los clientes evaluados mediante la aplicación.

Predicción de abandono de clientes

Backend conectado -- modelo: xgboost_refinado.joblib

Predicción individual Ficha de cliente **Explicabilidad global** Monitoreo

Importancia global de variables (SHAP)

Calculada una sola vez sobre el conjunto de validación en notebook/07_explicabilidad_shap.ipynb -- no se recalcula en la app.

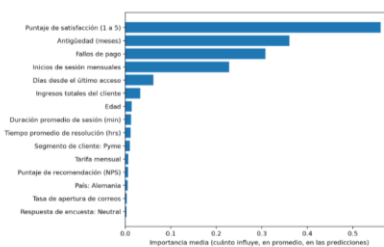


Figura 18. Importancia global de variables en la pestaña Explicabilidad global.

Fuente: elaboración propia.

Predicción de abandono de clientes

Backend conectado -- modelo: xgboost_refinado.joblib

Predicción individual Ficha de cliente Explicabilidad global **Monitoreo**

Actividad de la aplicación

Predicciones registradas

4

Tiempo de respuesta promedio

12.5 ms

% en banda Alta

75%



Ver eventos registrados

	Fecha y hora	Origen	Probabilidad	Banda de riesgo	Umbral	Tiempo de respuesta (ms)
0	25/07/2026 22:45	Formulario	0.8341	Alto	0.56	13.1369
1	25/07/2026 22:46	Ficha de cliente	0.2313	Bajo	0.56	13.0472
2	25/07/2026 22:46	Ficha de cliente	0.6732	Alto	0.56	11.329
3	25/07/2026 22:50	Ficha de cliente	0.5919	Alto	0.56	12.6846

Figura 19. Pestaña de monitoreo, con actividad de la aplicación y desempeño validado del modelo.

Fuente: elaboración propia.

V. Resultados

Tabla 13. Comparación de tres modelos de ML y tres arquitecturas de DL en validación

Modelo	Accuracy	Precisión	Recall	F1	ROC-AUC	PR-AUC
Regresión logística + ROS	0,705	0,215	0,712	0,330	0,751	0,221
Random Forest + ROS	0,776	0,278	0,752	0,406	0,818	0,302
XGBoost + ROS	0,737	0,259	0,843	0,396	0,826	0,333
MLP superficial	0,728	0,216	0,634	0,322	0,763	0,237
MLP profunda	0,683	0,208	0,752	0,326	0,772	0,252
MLP residual	0,722	0,218	0,667	0,329	0,763	0,237

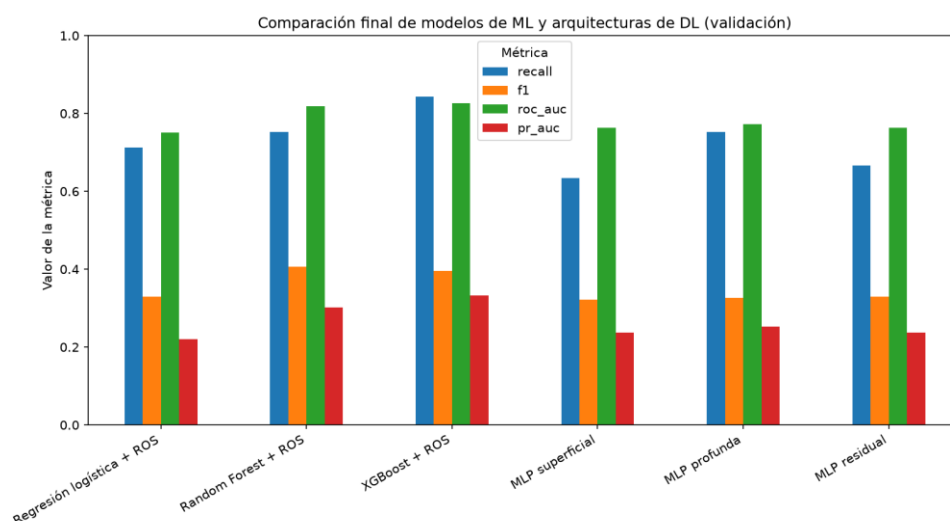


Figura 20. Comparación de los seis candidatos en validación.

Fuente: elaboración propia.

Las redes neuronales no superaron a los principales modelos de machine learning. Random Forest + ROS obtuvo la mayor precisión y el mejor F1. XGBoost + ROS alcanzó el mayor recall, ROC-AUC y PR-AUC, por lo que presentó el perfil más alineado con el objetivo de anticipar abandonos.

5.1 Evaluación de XGBoost + ROS en prueba

Tabla 14. Métricas del modelo seleccionado sobre 1.500 casos de prueba

Modelo	Accuracy	Precisión	Recall	F1	ROC-AUC	PR-AUC
XGBoost + ROS	0,719	0,232	0,758	0,355	0,781	0,300

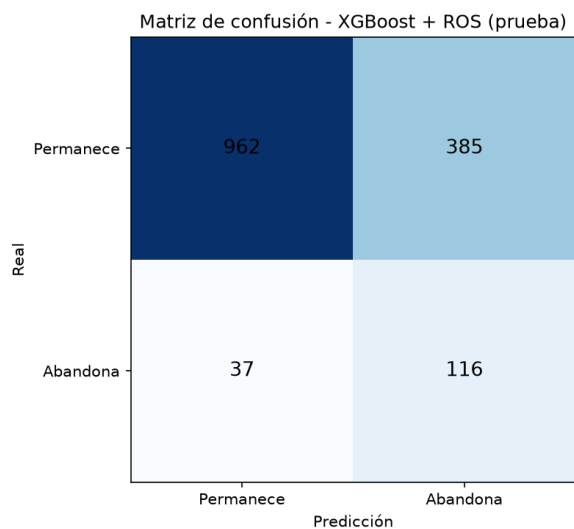


Figura 21. Matriz de confusión de XGBoost + ROS en prueba.

Fuente: elaboración propia.

El modelo detectó correctamente a 116 de los 153 clientes que abandonaron, alcanzando un recall de 0,758. La matriz contiene 962 verdaderos negativos, 116 verdaderos positivos, 385 falsos positivos y 37 falsos negativos. En términos simples, identifica aproximadamente tres de cada cuatro abandonos reales, pero genera una cantidad importante de alertas falsas.

El recall disminuyó desde 0,843 en validación a 0,758 en prueba. Esta diferencia muestra la dificultad de mantener el desempeño sobre datos completamente nuevos y refuerza la necesidad de monitoreo.

5.2 Resultados del refinamiento

El modelo definitivo (XGBoost + ROS con los hiperparámetros y el umbral 0,56 de la sección 4.5.3) se evaluó sobre el conjunto de prueba como lectura final adoptada para el proyecto. Cabe transparentar que, durante el desarrollo, también se obtuvo una lectura exploratoria de prueba con el umbral F1-máximo (0,66) antes de descartarlo por su bajo recall en validación; esa lectura no participó en la selección del umbral definitivo, que se decidió exclusivamente con evidencia de validación (sección 4.5.3). Los resultados de la configuración adoptada mejoran de forma pequeña pero consistente sobre el baseline original en prácticamente todas las métricas: accuracy 0,721 (antes 0,719), precision 0,234 (antes 0,232), recall 0,765 (antes 0,758), F1 0,359 (antes 0,355), ROC-AUC 0,782 (antes 0,781) y PR-AUC 0,302 (antes 0,300).

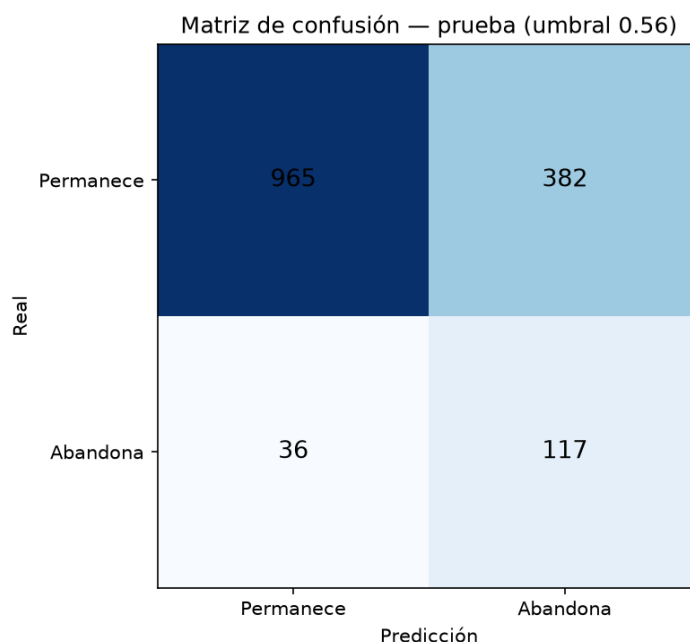


Figura 22. Matriz de confusión de XGBoost + ROS refinado en el conjunto de prueba (umbral 0,56).

Fuente: elaboración propia.

La mejora no es grande, pero es consistente y, sobre todo, no compromete el recall: el modelo refinado detecta una proporción de abandonos igual o mayor que el original, que es el criterio que más le importa al proyecto. El hallazgo metodológico más relevante fue detectar mediante validación que optimizar F1 sin restricciones producía un modelo que se veía mejor en términos agregados, pero fallaba en la prioridad real del negocio (ver sección 4.5.3).

5.3 Interpretación de resultados con SHAP

Esta sección interpreta las predicciones del modelo definitivo (XGBoost + ROS refinado, sección 4.5.3) mediante SHAP (SHapley Additive exPlanations; SHAP, s. f.), en dos niveles: qué variables influyen más en promedio sobre el modelo (importancia global) y cómo se explica la predicción de clientes concretos (explicaciones locales). El detalle completo del análisis está en `notebook/07_explicabilidad_shap.ipynb`.

Nota metodológica: se intentó calcular los valores SHAP en espacio de probabilidad, para leerlos directamente como puntos de probabilidad de abandono. Ese modo no es compatible con los splits categóricos nativos que usa la versión actual de XGBoost, por lo que los valores presentados a continuación están en espacio de margen (log-odds): el signo indica si la variable empuja la predicción hacia el abandono (positivo) o hacia la permanencia (negativo), y la magnitud indica qué tan fuerte es ese empuje en relación con las demás variables del mismo caso. No deben leerse como puntos porcentuales de probabilidad.

La Tabla 15 y la Figura 23 muestran las variables con mayor importancia global, medida como el promedio del valor absoluto de SHAP sobre el conjunto de validación. `csat_score`, `tenure_months`, `payment_failures` y `monthly_logins` concentran la mayor parte de la importancia, en ese orden — las mismas cuatro variables que ya se habían identificado en el análisis exploratorio (sección 4.1.6) como las de mayor diferencia (Cohen's d) entre clientes que permanecen y abandonan. Esta coincidencia muestra que las señales aprendidas son coherentes con el análisis exploratorio, aunque no descarta relaciones espurias ni implica causalidad.

Tabla 15. Importancia global de variables según SHAP (promedio de $|\text{valor}|$ en validación).

Variable	Importancia media SHAP
<code>csat_score</code>	0,561
<code>tenure_months</code>	0,361
<code>payment_failures</code>	0,308
<code>monthly_logins</code>	0,228
<code>last_login_days_ago</code>	0,062
<code>total_revenue</code>	0,033
<code>age</code>	0,014
<code>avg_session_time</code>	0,013

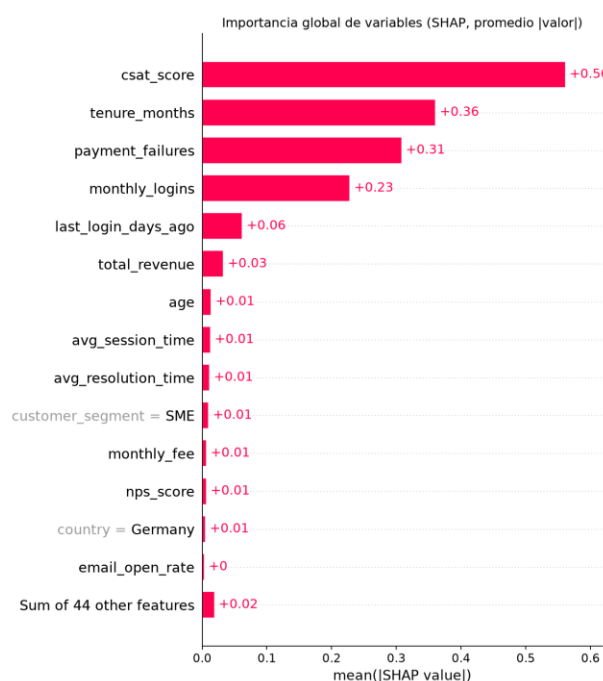


Figura 23. Importancia global de variables según SHAP.

Fuente: elaboración propia.

La Figura 24 complementa esta vista agregada: muestra, para cada variable, cómo se distribuye su efecto entre los clientes del conjunto de validación. Valores bajos de `csat_score`, `tenure_months` y `monthly_logins`, junto con valores altos de `payment_failures`, empujan sistemáticamente la predicción hacia el abandono; sus opuestos empujan hacia la permanencia.

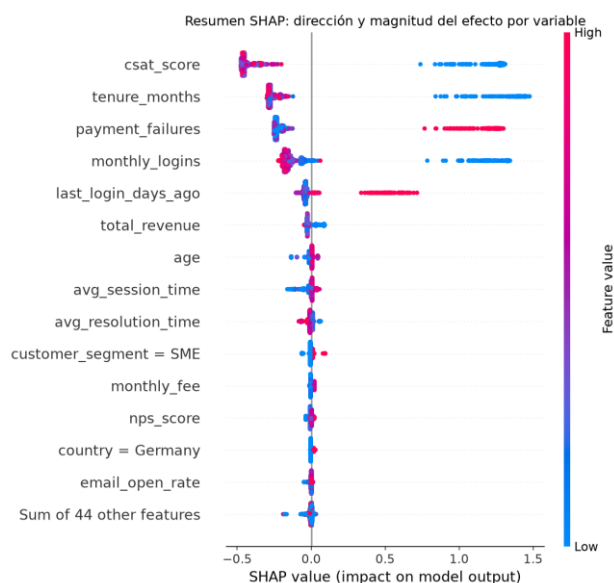


Figura 24. Resumen SHAP: dirección y magnitud del efecto de cada variable.

Fuente: elaboración propia.

Para ilustrar cómo se combinan estas señales en clientes concretos, se seleccionaron tres casos del conjunto de prueba: un verdadero positivo (abandono detectado correctamente), un falso positivo (alerta de riesgo en un cliente

que finalmente permaneció) y un verdadero negativo confiado (probabilidad baja en un cliente que efectivamente permaneció). Los tres casos usan el mismo umbral de decisión (0,56) definido en la sección 4.5.3.

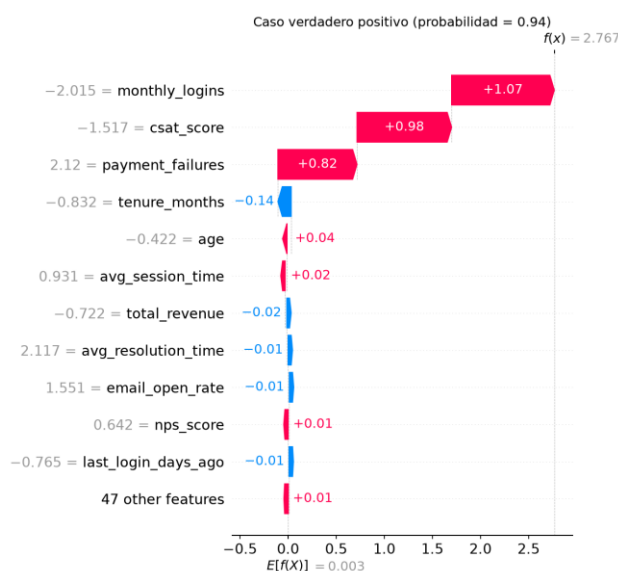


Figura 25. Explicación local de un verdadero positivo (probabilidad = 0,94).

Fuente: elaboración propia.

Este cliente fue detectado correctamente como alto riesgo (probabilidad 0,941). Las variables que más empujaron la predicción hacia el abandono fueron monthly_logins, csat_score y payment_failures, todas en el sentido esperado (accesos y satisfacción muy por debajo del promedio, fallos de pago muy por encima): es el perfil que el proyecto busca detectar

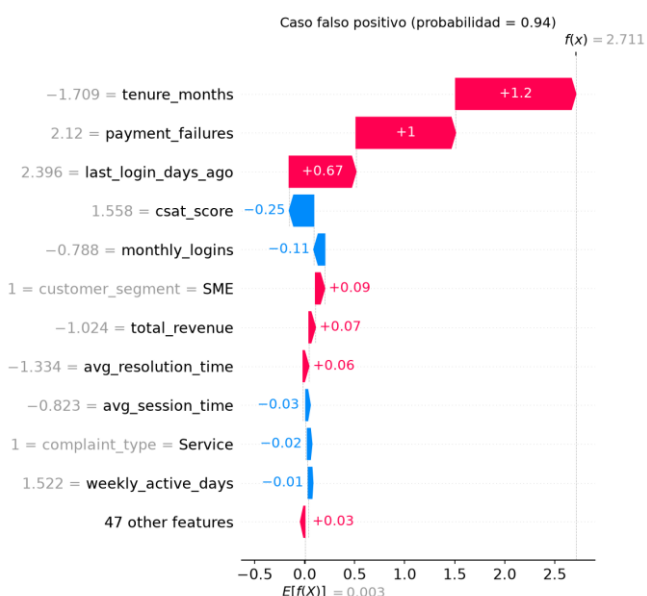


Figura 26. Explicación local de un falso positivo (probabilidad = 0,94).

Fuente: elaboración propia.

El modelo asignó a este cliente una probabilidad casi idéntica (0,938), pero permaneció: es exactamente el tipo de alerta falsa que explica la precisión baja del modelo (0,234 en prueba, Tabla 14). Las señales de riesgo —poca antigüedad, fallos de pago y muchos días sin acceder— pesaron más que su satisfacción por encima del promedio, que empujó en sentido contrario pero con menos fuerza. En otras palabras: el modelo ponderó más la inactividad y los problemas de pago que la buena satisfacción declarada, y en este caso se equivocó.

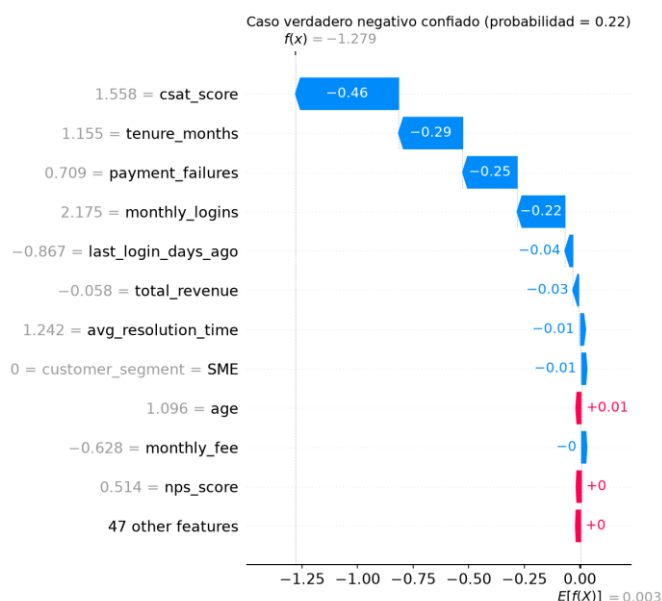


Figura 27. Explicación local de un verdadero negativo confiado (probabilidad = 0,22).

Fuente: elaboración propia.

Este cliente fue clasificado con probabilidad baja (0,218) y efectivamente permaneció. Su satisfacción muy por encima del promedio fue, de lejos, la señal más fuerte hacia la permanencia, reforzada por una antigüedad y una cantidad de accesos también por encima del promedio: el perfil inverso al primer caso.

Estas explicaciones muestran qué variables usó el modelo para cada predicción, es decir, asociaciones aprendidas de los datos de entrenamiento; no son evidencia de relaciones causales. Por ejemplo, no puede afirmarse que mejorar artificialmente el csat_score de un cliente vaya a causar que deje de abandonar: ambas variables podrían responder a una causa común, como la calidad del soporte técnico. Para decisiones de retención, estas explicaciones sirven para priorizar y entender el riesgo detectado por el modelo, no como receta causal.

5.4 Evaluación del prototipo y cumplimiento de requisitos

El backend y el frontend se probaron de tres formas independientes. Primero, con 43 pruebas automatizadas (pytest) que cubren validación de entradas (incluyendo límites objetivamente imposibles y advertencias por rango histórico), cálculo de la predicción, agregación correcta de SHAP para variables categóricas, clasificación del riesgo, registro de monitoreo (éxitos y errores) y los seis endpoints de la API — todas en verde. Segundo, comparando la probabilidad que entrega la API contra la que se obtiene cargando los mismos artefactos directamente, sin pasar por la aplicación, para 20 clientes reales del conjunto de prueba: la diferencia fue 0,0 en los 20 casos, confirmando que la app no introduce ningún desvío respecto al modelo entrenado. Tercero, con pruebas de extremo a extremo en un navegador

real (Playwright): se completó el formulario, se consultó la ficha de un cliente de riesgo bajo y de uno de riesgo alto, y se verificó que el modal se abre con el resultado correcto, que se cierra sin perder los datos ingresados, y que no se genera ningún error en la consola del navegador.

Tiempos de respuesta. Sobre 100 predicciones reales, el tiempo de cómputo del backend (transformación de datos, cálculo de probabilidad y explicación SHAP) promedió 9,5 ms, con una mediana de 9,3 ms y un percentil 95 de 11,1 ms — muy por debajo del requisito de responder en pocos segundos.

Manejo de errores. Una entrada inválida (categoría desconocida, campo faltante) devuelve un mensaje de error específico en español, sin interrumpir la sesión; un cliente inexistente en la ficha devuelve un error claro en vez de un resultado vacío; y si el backend no está disponible, la app lo indica explícitamente en vez de fallar en silencio.

Monitoreo. Descrito en la sección 4.7.4: cada predicción queda registrada con su probabilidad, banda de riesgo y tiempo de respuesta, sin datos crudos del cliente, y la pestaña "Monitoreo" (Figura 19) permite revisar esta actividad en cualquier momento.

La Tabla 16 resume el cumplimiento de los requisitos directamente relacionados con el prototipo (frontend, backend, explicabilidad y monitoreo); el estado completo de todos los requisitos del proyecto, incluyendo los de fases anteriores, se actualizó en las Tablas 8 y 9 (sección 4.4).

Tabla 16. Cumplimiento final de los requisitos del prototipo.

Requisito	Evidencia de cumplimiento	Estado
RF-08: formulario de predicción y ficha de cliente.	Formulario de predicción individual y ficha de cliente Figura 16 / sección 4.7.3.	Completado
RF-09: resultado con probabilidad y banda de riesgo.	Barra de probabilidad con banda de riesgo y umbral (Figura 17).	Completado
RF-10: explicación local y explicabilidad global.	Factores reales por dirección + gráfico técnico (Figuras 17 y 18).	Completado
RF-11: pestaña de monitoreo.	monitoring.py y pestaña Monitoreo (Figura 19).	Completado
Usabilidad: interfaz clara para no técnicos.	Etiquetas y valores en español, agrupación lógica, límites por campo.	Completado
Explicabilidad: importancia global y local con SHAP.	Pestaña Explicabilidad global (Figuras 17 y 18) + explicación local por predicción.	Completado
Escalabilidad: separar preprocesamiento, inferencia e interfaz.	Backend y frontend como procesos HTTP independientes (Figura 15).	Completado
Monitoreabilidad: registrar volumen, riesgo y distribución.	Pestaña Monitoreo: conteo, distribución por banda, tiempo de respuesta.	Completado
Rendimiento: responder en pocos segundos en CPU.	9,5 ms promedio sobre 100 predicciones reales.	Completado
Seguridad: no guardar información sensible.	Prueba automatizada que verifica que no se filtran variables crudas.	Completado
Ética: apoyar, no automatizar, decisiones sobre clientes.	La app entrega probabilidad y explicación; no ejecuta ninguna acción sobre el cliente.	Completado

VI. Propuesta de mejoras

1. Validar con datos reales y monitorear drift. El dataset es sintético y puede no representar completamente una organización. Se propone validar con datos reales anonimizados y monitorear la distribución de variables, probabilidades y métricas a lo largo del tiempo.
2. Calibrar probabilidades y definir umbrales mediante costos reales. La precisión de 0,234 evidencia una cantidad importante de falsos positivos. Como mejora futura, se propone calibrar las probabilidades con un conjunto independiente y seleccionar el umbral considerando el costo de intervenir innecesariamente frente al costo de no detectar un abandono.
3. Esquema de entrada configurable para otros negocios. El esquema de las 30 variables de entrada (backend/schema.py) está codificado específicamente para este dataset; adaptar el sistema a otra empresa u otro plan de suscripción hoy requeriría modificar el código. Como mejora futura, se propone externalizar el esquema (variables, rangos, categorías válidas) a un archivo de configuración por cliente, para que la aplicación pueda reutilizarse en otro rubro sin reescribir el backend.
4. Predicción por lotes y vista de cartera priorizada. El prototipo actual predice un cliente a la vez (formulario o ficha individual). Como mejora futura, se propone permitir cargar un archivo CSV con varios clientes y mostrar una vista de cartera que los ordene por nivel de riesgo, para apoyar la priorización cuando el equipo de retención necesita revisar muchos casos a la vez, no solo uno.
5. Medir el impacto real de las acciones de retención. El sistema entrega una probabilidad y una explicación, pero no sabe si las acciones que el equipo de retención decide tomar a partir de esa información efectivamente reducen el abandono. Como mejora futura, y conectando con datos históricos reales de un CRM, se propone registrar qué clientes recibieron una intervención de retención y comparar su tasa de abandono real contra la de clientes similares que no la recibieron, para evaluar si la intervención marca una diferencia y reentrenar el modelo incorporando ese resultado.

VII. Código fuente en GitHub

El código, los notebooks, los artefactos y la aplicación se encuentran disponibles en el [repositorio del proyecto](#) (Vera et al., 2026).

La rama main contiene los notebooks 01 a 07 (calidad y EDA, preprocesamiento, modelamiento ML, balanceo y modelamiento DL, refinamiento de XGBoost y de la MLP profunda, y explicabilidad SHAP), además de los módulos backend/ (API de inferencia FastAPI) y frontend/ (aplicación Streamlit) del prototipo, y las pruebas automatizadas en tests/.

VIII. Bibliografía

Ahmad, A. K., Jafar, A., & Aljoumaa, K. (2019). Customer churn prediction in telecom using machine learning in big data platform. *Journal of Big Data*, 6, Article 28. <https://doi.org/10.1186/s40537-019-0191-6>

- Alotaibi, M. Z., & Haq, M. A. (2024). Customer churn prediction for telecommunication companies using machine learning and ensemble methods. *Engineering, Technology & Applied Science Research*, 14(3), 14572-14578. <https://doi.org/10.48084/etasr.7480>
- Arik, S. Ö., & Pfister, T. (2021). TabNet: Attentive interpretable tabular learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(8), 6679-6687. <https://doi.org/10.1609/aaai.v35i8.16826>
- Bhattacharjee, S., Thukral, U., & Patil, N. (2023). Early churn prediction from large scale user-product interaction time series. arXiv. <https://arxiv.org/abs/2309.14390>
- Borisov, V., Leemann, T., Seßler, K., Haug, J., Pawelczyk, M., & Kasneci, G. (2024). Deep neural networks and tabular data: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 35(6), 7499-7519. <https://doi.org/10.1109/TNNLS.2022.3229161>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32. <https://doi.org/10.1023/A:1010933404324>
- Burez, J., & Van den Poel, D. (2009). Handling class imbalance in customer churn prediction. *Expert Systems with Applications*, 36(3), 4626-4636. <https://doi.org/10.1016/j.eswa.2008.05.027>
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321-357. <https://doi.org/10.1613/jair.953>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794). Association for Computing Machinery. <https://doi.org/10.1145/2939672.2939785>
- Gorishniy, Y., Rubachev, I., Khrulkov, V., & Babenko, A. (2021). Revisiting deep learning models for tabular data. *Advances in Neural Information Processing Systems*, 34, 18932–18943. <https://proceedings.neurips.cc/paper/2021/hash/9d86d83f925f2149e9edb0ac3b49229c-Abstract.html>
- He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263-1284. <https://doi.org/10.1109/TKDE.2008.239>
- Imani, M., Abdolrazzagah-Nezhad, M., & Boushehri, A. (2025). Customer churn prediction: A systematic review of machine learning and deep learning approaches. *Machine Learning and Knowledge Extraction*, 7(3), 105. <https://doi.org/10.3390/make7030105>
- Maan, J., & Maan, H. (2023). Customer churn prediction model using explainable machine learning. arXiv. <https://arxiv.org/abs/2303.00960>
- Miah, A. (2025). Customer churn prediction business dataset [Data set]. Kaggle. <https://www.kaggle.com/datasets/miadul/customer-churn-prediction-business-dataset>
- Nogare, D., & Silveira, I. F. (2024). Experimentation, deployment and monitoring machine learning models: Approaches for applying MLOps. arXiv. <https://arxiv.org/abs/2408.11112>
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533-536. <https://doi.org/10.1038/323533a0>
- Shaikhsurab, M. A., & Magadum, P. (2024). Enhancing customer churn prediction in telecommunications: An adaptive ensemble learning approach. arXiv. <https://arxiv.org/abs/2408.16284>
- SHAP. (n.d.). *Welcome to the SHAP documentation*. <https://shap.readthedocs.io/en/latest/>

- Vafeiadis, T., Diamantaras, K. I., Sarigiannidis, G., & Chatzisavvas, K. C. (2015). A comparison of machine learning techniques for customer churn prediction. *Simulation Modelling Practice and Theory*, 55, 1-9. <https://doi.org/10.1016/j.simpat.2015.03.003>
- Vera, I., Vera, G., & Mallea, D. (2026). *Predicción de abandono de clientes (churn): ACIF104, Grupo 3* [Código fuente]. GitHub. <https://github.com/gveraprado-wited/acif104-churn-grupo3>